Efficient Construction of Threshold BBS+ Signatures and its Extensions

Yang Heng[1], Mengling Liu[1], Xingye Lu[1], Haiyang Xue[2], Zijian Bao[1], and Man Ho Au[1]*

[1] The Hong Kong Polytechnic University, Hong Kong, China
yang2023.heng@connect.polyu.hk, mengling.liu@connect.polyu.hk, xing-ye.lu@polyu.edu.hk, zibao@polyu.edu.hk, man-Ho-Allen.au@polyu.edu.hk
[2] Singapore Management University, Singapore
haiyangxc@gmail.com

Abstract. BBS+ signatures are widely adopted in privacy-preserving systems such as anonymous credentials and Direct Anonymous Attestation (DAA). To strengthen key security and eliminate single points of failure, threshold variants of BBS+ signatures have become increasingly important. However, existing constructions suffer from notable inefficiencies: some entail excessive communication overhead (e.g., DKL+23, S&P 2023), while others impose substantial computational costs and require additional interaction rounds (e.g., WMC24, NDSS 2024).
In this work, we present a novel and efficient three-round threshold BBS+ signature scheme from the Castagnos–Laguillaumie (CL) cryptosystem. Our construction achieves best communication–computation trade-offs than previous works. Specifically, compared to the four-round WMC24 scheme, our protocol reduces communication by 77.4% and demonstrates faster computation, with benchmarks indicating speedups of 10.6–16.6× in single-threading and 3.3–5.4× in multi-threading. Against the three-round protocol DKL+23, our scheme exhibits an asymptotic slowdown factor of 4×, but enhances communication by two orders of magnitude.
We further extend our techniques to threshold BBS signatures, Dodis-Yampolskiy verifiable random functions (DY VRFs), and multiplication protocols (DNP25 and LLZ+25, CCS'25). This yields: (1) a three-round threshold protocol for the original BBS scheme; (2) two-round threshold protocols for both DY VRFs (focusing on its oblivious variant) and the AGM-secure BBS variant; and (3) one fewer group element in broadcasts for the multiplication protocol with reduced ZKP costs via simplified relations.

Keywords: Threshold cryptography · BBS+ signatures.

1 Introduction

BBS signatures were first introduced by Boneh, Boyen, and Shacham in their group signature scheme [2] and later explicitly formulated as a standalone scheme

* Corresponding author: man-ho-allen.au@polyu.edu.hk

in [8]. Subsequently, Au, Susilo, and Mu devised the BBS+ signature scheme [1] as a provably secure version. The pioneering works [7, 8] by Camenisch and Lysyanskaya show that, BBS(+) signatures are amenable to several privacy-preserving techniques such as anonymous credentials [18, 23] and direct anonymous attestation (DAA) [5, 6]. Notably, BBS+ has been deployed in the Enhanced Privacy ID (EPID) system of Intel's SGX platform [5]. Recognizing its broad applicability, the Internet Research Task Force (IRTF) has initiated standardization efforts for BBS(+) schemes [30], underscoring their growing importance in practical cryptographic employments.

To strengthen key protection, a threshold BBS+ construction can be employed to mitigate the risk of a single point of failure. For instance, in conventional anonymous credential systems based upon BBS+, the signing key is controlled solely by the issuer, who generates credential signatures to authenticate users. If an adversary compromises the issuer, they gain access to the signing key, enabling the generation of unlimited fraudulent credentials. Furthermore, because these credentials are anonymous, such malicious activity remains undetectable and untraceable. A (t, n) threshold scheme [16] distributes the signing key among n parties, requiring the cooperation of at least t parties to produce a valid signature. This approach mitigates attacks by ensuring that an adversary must compromise at least t parties to forge a signature.

Thresholding BBS+ Signatures. The BBS+ signing algorithm takes as input the signing key $x \in \mathbb{Z}_q$ and a message $\mathbf{M} = (m_1, \ldots, m_\ell) \in \mathbb{Z}_q^\ell$, and generates a signature (A, e, s) as follows:

(1) Sample uniform nonces $e, s \xleftarrow{\$} \mathbb{Z}_q$.
(2) Compute the elliptic-curve group element

$$A := \frac{B}{x+e} \in \mathbb{G}_1,$$

where $B := G_1 + s \cdot H_1 + \sum_{i \in [\ell]} m_i \cdot H_{i+1} \in \mathbb{G}_1$, and $(G_1, H_1, \ldots, H_{\ell+1}) \in \mathbb{G}_1$.

The key challenge in thresholdizing BBS+ lies in how to securely perform operations (1) and (2) in a distributed manner, i.e., with the values x and/or e, s being secret-shared among the parties.

To perform the above operation securely, DKL+23 protocol [18] employs a commitment scheme to distributively generate e and s in an unbiased manner, and utilizes the Multiplicative-to-Additive conversions (MtA conversions, also known as oblivious linear evaluation) for the distributed computation of A. Here, MtA is a two-party protocol that converts multiplicative shares (a, b) into additive shares (α, β) such that $\alpha + \beta = ab$. While DKL+23 achieves remarkable speed—thanks to its OT-based MtA instantiations—it needs $\mathcal{O}(t)$ outgoing communication complexity per party due to pairwise MtA interactions. This leads to high concrete cost: for example, each party transmits roughly 670KiB in a 10-party setup.

Recently, WMC24 [31] introduced a threshold scheme based on threshold additively homomorphic encryption (TAHE). They cast threshold BBS+ signing as a secure multiparty computation (MPC) task and realize it via the

Cramer–Damgård–Nielsen (CDN) paradigm [14] instantiated with TAHE. Concretely, the generation of e and s and the computation of A are carried out using TAHE operations, while zero-knowledge proofs (ZKPs) attest to the correctness of these steps. Inspired by the efficient implementation [3] of the additively homomorphic CL encryption scheme [13], they instantiate TAHE over $\mathbb{Z}_q$ within the CL framework, while TAHE over elliptic-curve groups is realized via threshold ElGamal. The resulting protocol achieves $\mathcal{O}(1)$ outgoing communication and offers an improved communication–computation trade-off; for instance, it reduces per-party broadcast data to 3.45 KiB, albeit being roughly two orders of magnitude slower than DKL+23, taking 200.44 ms for 10 parties.

As summarized in Table 1, the existing threshold BBS+ schemes either incur significantly higher communication overhead per party (approximately 670 KiB in a 10-party setting, as in DKL+23) or demand substantially greater overall computational cost.

Motivation. Designing an efficient threshold BBS+ scheme for large-scale deployment—particularly in cryptocurrency settings where trust is paramount—remains a central challenge. As the number of signers (i.e., the parties participating in the signing execution) increases to strengthen security, existing solutions often incur substantial trade-offs between communication and computation. This raises the following fundamental research question:

> *Can we design a threshold BBS+ signature scheme that achieves a superior trade-off between communication and computation costs by leveraging cryptographic primitives better suited for threshold settings?*

1.1 Our Contributions

We address this question by proposing an efficient threshold BBS+ signature scheme that achieves a more favorable trade-off between computation and communication than both DKL+23 and WMC24.

- We present an efficient 3-round threshold BBS+ scheme combining a commitment scheme with CL KEM (a conceptual key encapsulation mechanism abstracted from threshold CL encryption to demonstrate our approach, but is not a secure KEM), matching DKL+23's round complexity while achieving $\mathcal{O}(1)$ communication like WMC24. Our protocol reduces WMC24's requirements from 9 to 2 broadcast class-group elements per signer and from $\sim 29t$ to $\sim 2t$ class-group exponentiations. While computationally more expensive than DKL+23's OT-based approach, our CL-KEM-based solution achieves a superior computation-communication trade-off (Table 1), with security proven in the simulation-based model.
- We extend our threshold techniques to both BBS signatures [2,29] and Dodis-Yampolskiy verifiable random functions [17] (denoted as DY VRFs), as shown in Section 4. This is because their signing/evaluation processes require only the nonce generation and/or computation of A. As a result, we obtain (1) a 3-round threshold protocol for the original BBS scheme [2], and (2) 2-round

threshold protocols for both DY VRFs (focusing on its oblivious evaluation) and the AGM-secure variant of BBS [29].

– We generalize our CL KEM techniques to recent multiplication protocols DNP25 [15] and LLZ+25 [25], whose underlying constructions—both derived from the CL framework—share compatible logical foundations. As detailed in Section 5, our CL KEM design can optimize these protocols, yielding concrete efficiency gains: eliminating one class-group element broadcast and reducing ZKP computation overhead.
– We provide a Rust implementation of our threshold BBS+ protocol and present benchmarks in Section 6, including performance (runtime and bandwidth) comparisons with WMC24 and DKL+23. Since no implementation or experimental data were provided in WMC24, we re-implemented their BBS+ scheme in Rust. Our evaluation is conducted in the same environment as DKL+23, and their results are taken directly from published data. Extensive testing across 2–20 parties reveals significant efficiency improvements:
  - We conduct comparisons with WMC24 to focus on computational time differences. Under multi-threading, our protocol achieves 3.3-5.4× speedup. Single-threaded analysis reveals accelerating gains approaching 16.6× with increasing party count, validating our theoretical overhead reduction.
  - We compare our protocol with DKL+23 in the multi-threaded setting, as their paper does not provide single-threaded results. Our evaluation shows that while our protocol's runtime is 4-16× slower, it achieves ∼100× better communication efficiency.

	Latency(Multi/Single threaded)	Bandwidth	Round	Note
DKL+23 [18]	∼6.3ms / -	678.14KiB, $\mathcal{O}(t)$	3	-
WMC24 [31]	200.4ms / 3112.7ms	3.45KiB, $\mathcal{O}(1)$	4	IA,robust
Our work	47.9ms / 199.5ms	0.78KiB, $\mathcal{O}(1)$	3	-

Table 1: A comparison of the signing protocols between our work and existing works. Bandwidth refers to the communication cost per party participating in the signing process. When not explicitly stated, communication cost typically denotes outgoing communication cost. IA = Identifiable Abort. The robust property ensures the protocol terminates successfully as long as a threshold of honest parties participate in each round. In contrast, non-robust protocols require the same subset of parties to remain active throughout. Data is reported for $t = 10$; $n \geq t$ shows no impact on results.

1.2 Current Threshold BBS+ Signatures

Before introducing our approach, we review the relevant background and prior works in this subsection.

The Framework of Threshold BBS+. We begin by outlining the framework adopted by DKL+23 [18] and WMC24 [31]. It comprises four phases:

1. **Key Generation**: All parties jointly generate the verifying key and secret-share the signing key x.
2. **Nonce Generation**: Signers jointly sample the nonces e and s.
3. **Signature Components**: The signers jointly compute $R := \gamma B$ and $u := \gamma(x + e)$, where γ is a secret value.
4. **Signature Derivation**: Set $A := R/u$ and output the signature (A, e, s).

Phase 1 is executed once to establish the verifying-signing key pair, whereas Phases 2–4 are repeated for each message to sign a new signature. In the subsequent content, we discuss the key differences between DKL+23 and WMC24, which arise in Phases 2 and 3.

DKL+23. The execution of Phases 2 and 3 is divided into three subtasks. Each signer P_i, for $i \in \mathcal{T}$, proceeds as follows, where $\mathcal{T}$ denotes the signer index set:

1. **Derive $(\boldsymbol{e = \sum_{j\in\mathcal{T}} e_j, s = \sum_{j\in\mathcal{T}} s_j})$**: Sample and commit to $e_i, s_i \xleftarrow{\$} \mathbb{Z}_q$, then release (e_i, s_i) to construct (e, s).
2. **Derive $\boldsymbol{u = \gamma(x+e) = \big(\sum_{j\in\mathcal{T}} \gamma_j\big)\big(\sum_{k\in\mathcal{T}} x_k + e\big)}$**: Before interacting with other signers, (1) sample $\gamma_i \xleftarrow{\$} \mathbb{Z}_q$ and $\gamma = \sum_{j\in\mathcal{T}} \gamma_j$ is implicitly defined, and (2) compute the additive share x_i from the Phase 1 output so that $\sum_{k\in\mathcal{T}} x_k = x$. Next, initiate $2(|\mathcal{T}|-1)$ instances of MtA with other signers to distributively compute $\gamma_i x_j$ and $\gamma_j x_i$. Once e is available, sum all MtA outputs together with the local term $\gamma_i(x_i + e)$ to obtain u_i. Then broadcast u_i to set $u = \sum_{j\in\mathcal{T}} u_j$.
3. **Derive $\boldsymbol{R = \sum_{j\in\mathcal{T}} R_j}$**: Broadcast $R_i = \gamma_i \cdot B$ to construct R.

As outlined above, each party performs $2(|\mathcal{T}| - 1)$ instances of MtA, which dominate the signing cost, and communication scales linearly in $|\mathcal{T}|$. Although switching to an AHE-based MtA (e.g., Paillier, with approximately 3.34 KiB and 175 ms per instance [32]) can substantially reduce the concrete communication overhead, the asymptotic complexity remains unchanged. The protocol (for the signing execution, excluding Phase 1) completes in 3 rounds: two rounds to construct (e, s) and invoke MtA, followed by one round to broadcast (u_i, R_i).

WMC24. WMC24 partitions the same three subtasks as DKL+23, but executes them using TAHE. Specifically, subtasks 1 and 2 are realized with TAHE over $\mathbb{Z}_q$, while subtask 3 is realized with TAHE over $\mathbb{G}_1$. The resulting protocol attains constant per-signer communication complexity. To clarify the source of this property, we first outline the TAHE mechanism. We write $\mathsf{Enc}(m)$ for an encryption of m, and use

$$(a \odot \mathsf{Enc}(m)) \oplus \mathsf{Enc}(b) = \mathsf{Enc}(am + b)$$

to denote the homomorphic encryption of $am+b$ (where "$\odot$" and "$\oplus$" respectively denotes the homomorphic scaling and addition). In addition, TAHE supports threshold decryption of $\mathsf{Enc}(m)$ as follows:

– Each signer P_i computes a partial decryption pd_i and broadcasts it.
– Each P_i recovers m from $\{pd_j\}_{j\in\mathcal{T}}$.

Deriving e is straightforward: each signer P_i broadcasts $\mathsf{Enc}(e_i)$ and concludes $\mathsf{Enc}(e) = \bigoplus_{j\in\mathcal{T}} \mathsf{Enc}(e_j)$, then runs threshold decryption to obtain e. The same logic applies to deriving s and R.

The derivation of u proceeds as:

1. Each signer P_i computes and broadcasts $\mathsf{Enc}(x_i)$, then concludes $\mathsf{Enc}(x) = \bigoplus_{j\in\mathcal{T}} \mathsf{Enc}(x_j)$. Note that $\mathsf{Enc}(x)$ can be reused across signing executions; hence, this step can be moved to Phase 1 and performed once.
2. Each P_i locally computes $\mathsf{Enc}(\gamma_i x) = \gamma_i \odot \mathsf{Enc}(x)$. Once e is available, P_i computes and broadcasts $\mathsf{Enc}(\gamma_i(x+e)) := \mathsf{Enc}(\gamma_i x) \oplus \mathsf{Enc}(\gamma_i e)$, and concludes $\mathsf{Enc}(\gamma(x+e)) = \bigoplus_{j\in\mathcal{T}} \mathsf{Enc}(\gamma_j(x+e))$.
3. All signers perform threshold decryption to obtain $u = \gamma(x+e)$.

The constant communication cost follows directly, since each signer broadcasts only a constant number of messages. The overall computational cost is dominated by TAHE-related operations, especially the ZKPs used to validate TAHE messages, as each proof must be verified by all signers. The protocol completes in 4 rounds: the first 2 rounds construct (e, s), followed by 2 rounds to construct (u, R).

Remark 1. Although subtask 3 completes in round 4 due to its dependency on s, we observed that the broadcast of $\mathsf{Enc}(\gamma_i(x+e))$ can be advanced by one round, enabling u to be derived in round 3. Concretely, in step 2 of subtask 2 we set $\mathsf{Enc}(\gamma_i e) := \gamma_i \odot \mathsf{Enc}(e)$, where $\mathsf{Enc}(e)$ is available one round earlier than the finalized value e; this allows $\mathsf{Enc}(\gamma_i(x+e))$ to be computed and broadcast in round 2.

The Mechanisms to Achieve Security. WMC24 inherits its security from the CDN paradigm. For DKL+23, the construction of (e, s) is guaranteed to be unbiased due to the commitment scheme, while the rationale for how (R_i, u_i) securely derives A is more involved. We briefly outline the idea as follows. DKL+23 treats γ as a MAC key that authenticates the input B to produce the tag $R = \gamma \cdot B$. Since γ is hidden from all signers, direct tag verification is impossible. However, because γ is implicitly determined by $\frac{u}{x+e}$, verifying the signature $\left(\frac{R}{u}, e, s\right)$ implicitly validates R, thereby making misbehavior detectable.

1.3 Technique Overview

Our protocol builds on the established threshold BBS+ framework, partitioning Phases 2 and 3 into three subtasks. The core innovation lies in subtask 2, where we redesign the CL-based TAHE process. Specifically, our modified process eliminates broadcasting half part of individual ciphertexts by embedding it directly into partial decryptions, simultaneously reducing communication costs and computation costs (by eliminating associated ZKPs). Additionally, we simplify ZKP relations for the homomorphic scaling operation $\odot$ by leveraging the

algebraic structure of CL's underlying group. To clarify this divergence from classical CL-based TAHE, we abstract our approach using Key Encapsulation Mechanism (KEM) syntax—termed CL KEM. We emphasize that CL KEM is not a cryptographically secure KEM, but rather a conceptual framework reinterpreting threshold CL encryption within a KEM/DEM (i.e., data encapsulation mechanism) paradigm.

Background: CL-Based TAHE. The CL encryption scheme operates over a class group $\langle \mathfrak{g}_\mathfrak{q} \rangle \times \langle \mathfrak{f} \rangle$, where $\mathfrak{g}_\mathfrak{q}$ has order s' and $\mathfrak{f}$ has order q. Note that s' is coprime to q, implying $\langle \mathfrak{g}_\mathfrak{q} \rangle \cap \langle \mathfrak{f} \rangle = \{1\}$. The standard CL encryption scheme operates as the following:

- Key Generation: Generating a key pair (sk, pk) satisfies $pk = {\mathfrak{g}_\mathfrak{q}}^{sk}$.
- Encryption: To encrypt a message $m \in \mathbb{Z}_q$, let r be randomness:

$$\mathsf{CL.Enc}(m) := (\phi_K, \overline{m^K}), \quad \phi_K := {\mathfrak{g}_\mathfrak{q}}^r, \quad K := pk^r, \quad \overline{m^K} := K \cdot \mathfrak{f}^m,$$

- Decryption: Since $K = pk^r = \phi_K^{sk}$, the decryption proceeds as:

$$m := \mathsf{CL.Solve}\left(\frac{\overline{m^K}}{\phi_K^{sk}}\right) = \mathsf{CL.Solve}\left(\frac{K \cdot \mathfrak{f}^m}{\phi_K^{sk}}\right) = \mathsf{CL.Solve}(\mathfrak{f}^m),$$

  where $\mathsf{CL.Solve}$ efficiently solves discrete logarithms (DL) in $\langle \mathfrak{f} \rangle$.

CL homomorphic addition **($\oplus$)**. Given ciphertexts $(\phi_{K_1}, \overline{m_1^{K_1}})$ and $(\phi_{K_2}, \overline{m_2^{K_2}})$, their sum encrypting $m_1 + m_2$ is:

$$(\phi_{K_3}, \overline{m_1 + m_2^{K_3}}) := (\phi_{K_1}, \overline{m_1^{K_1}}) \oplus (\phi_{K_2}, \overline{m_2^{K_2}}) = (\phi_{K_1} \cdot \phi_{K_2}, \overline{m_1^{K_1}} \cdot \overline{m_2^{K_2}}),$$

where $K_3 = K_1 \cdot K_2$.

CL homomorphic scaling **($\odot$)**. Given a CL ciphertext $(\phi_K, \overline{m^K})$, the conventional way to encrypt am for $a \in \mathbb{Z}_q$ via $\odot$ is

$$(\phi_{K'}, \overline{am^{K'}}) := a \odot (\phi_K, \overline{m^K}) = (\phi_K^a {\mathfrak{g}_\mathfrak{q}}^r, \left(\overline{m^K}\right)^a pk^r),$$

where r is fresh random scalar. This ensures the ciphertext is indistinguishable from a direct encryption of am, thereby concealing the plaintext. The ZKP authenticating this operation contains two scalars in the witness: a and r.

Remark 2. Some works (e.g., [10,11]) omit fresh randomness in the scaling step because their broadcast ciphertexts take the form $\mathsf{CL.Enc}(am+b) = \mathsf{CL.Enc}(am) \oplus \mathsf{CL.Enc}(b)$, with the randomness injected in $\mathsf{CL.Enc}(b)$. $\mathsf{CL.Enc}(am + b)$'s distribution is identical whether the fresh randomness is placed in $\mathsf{CL.Enc}(am)$ or in $\mathsf{CL.Enc}(b)$. In our setting, however, we require the security of the standalone component $\mathsf{CL.Enc}(am)$, so the fresh randomness must be injected in $\mathsf{CL.Enc}(am)$.

Threshold decryption in CL-Based TAHE. Assume the secret key sk is additively shared among signers, i.e., P_i holds sk_i and $\sum_{j\in\mathcal{T}} sk_j = sk$.[3] For a common ciphertext $(\phi_K, \overline{m^K})$, each P_i computes a partial decryption $pd_i := \phi_K^{sk_i}$. The message is recovered as:

$$m := \mathsf{CL.Solve}\left(\frac{\overline{m^K}}{\prod_{j\in\mathcal{T}} pd_j}\right) = \mathsf{CL.Solve}\left(\frac{K\mathfrak{f}^m}{\phi_K^{sk}}\right).$$

A toy example workflow of using CL-Based TAHE. Consider two signers P_1 and P_2, holding ciphertexts $(\phi_{K_1}, \overline{m_1^{K_1}})$ and $(\phi_{K_2}, \overline{m_2^{K_2}})$ respectively. To jointly compute $m_1 + m_2$:

1. Homomorphic Addition: Each P_i broadcasts their ciphertext; the combined ciphertext $(\phi_{K_3}, \overline{m_1 + m_2^{K_3}})$ is computed via $\oplus$.
2. Threshold Decryption: Each P_i broadcasts pd_i. The sum $m_1 + m_2$ is derived as $\mathsf{CL.Solve}\left(\frac{\overline{m_1+m_2^{K_3}}}{\prod_{j\in\mathcal{T}} pd_j}\right)$ for $\mathcal{T} = \{1, 2\}$.

Each P_i broadcasts three class-group elements: ϕ_{K_i}, $\overline{m_i^{K_i}}$, and pd_i.

Our Observation. We observe that, we can reduce the number of broadcast elements per party in our toy example, by embedding $\overline{m_i^{K_i}}$ directly into partial decryptions pd_i. To formalize this, we introduce *CL KEM*—a syntactic framework reinterpreting threshold CL. This framework motivates our modification to the homomorphic scaling operation: combining witness scalars (a, r) into $a + qr$ by leveraging CL's class group properties. Simplifying the ZKP relation reduces computation costs, particularly in threshold settings where broadcast ZKPs are verified by $t - 1$ parties (i.e., signers in our context). Both techniques apply to subtask 2, allowing lower overhead.

Our Method—Reinterpreting threshold CL as CL KEM. In the classic KEM/DEM paradigm for public-key encryption (PKE), the KEM first encapsulates a random symmetric key K into a component ϕ using public key pk, and then the DEM encrypts the message m with key K as c, resulting in the ciphertext (ϕ, c). We noted that, the CL ciphertext structure naturally embodies the KEM/DEM paradigm: component ϕ encapsulates the symmetric key $K = \phi^{sk}$, while $\overline{m^K} = K \cdot \mathfrak{f}^m$ encrypts message m under K. Consequently, CL ciphertexts can be reinterpreted as pairs of encapsulated keys and symmetric ciphertexts, leading us to term this construction *CL KEM*.

Reorganizing the toy example workflow using CL KEM. Our reorganization performs the threshold decryption on a common encapsulation rather

[3] while in the actual protocol P_i computes an additive share sk_i' of $\Delta^2 sk$, where Δ^2 is a constant; see Fig. 8

than a full CL ciphertext. Each P_i first broadcasts ϕ_{K_i} to form the common encapsulation $\phi_{K_3} = \prod_{j\in\mathcal{T}} \phi_{K_j}$, then broadcasts pd_i redefined as $\frac{\overline{m_i^{K_i}}}{\phi_{K_3}^{sk_i}}$,[4] yielding:

$$m_1+m_2 := \mathsf{CL.Solve}\left(\prod_{j\in\mathcal{T}} pd_j\right) = \mathsf{CL.Solve}\left(\prod_{j\in\mathcal{T}} \frac{\overline{m_j^{K_j}}}{\phi_{K_3}^{sk_j}}\right) = \mathsf{CL.Solve}\left(\frac{K_3 \cdot \mathfrak{f}^m}{\phi_{K_3}^{sk}}\right).$$

In this reorganization, each P_i broadcasts only two class group elements (ϕ_{K_i} and pd_i), reducing the original requirement of three.

Extending the reorganization to subtask 2. This reorganization naturally extends to deriving u for subtask 2 over a general index set $\mathcal{T}$, where $u = \sum_{j\in\mathcal{T}} \gamma_i(x+e)$, thereby lowering communication and computational costs. This reorganization extends naturally to computing $u = \sum_{j\in\mathcal{T}} \gamma_i(x+e)$ for arbitrary index sets $\mathcal{T}$, reducing communication costs. Crucially, ZKP for $\mathsf{CL.Enc}(\gamma_i(x+e))$ simplifies to verifying only its encapsulation component, which saves computation costs. In addition, we do not append ZKPs for the partial decryptions that embeds the symmetric ciphertext component, and we justify this in later content. Specifically, we would like to mention that the encapsulation element of $\mathsf{CL.Enc}(\gamma_i(x+e)) = \mathsf{CL.Enc}(\gamma_i x) \oplus \mathsf{CL.Enc}(\gamma_i e)$ can be computed locally without knowing e. By setting $\mathsf{CL.Enc}(\gamma_i e) = (1, \mathfrak{f}^{\gamma_i e})$, the encapsulation element of $\mathsf{CL.Enc}(\gamma_i(x+e))$ coincides with that of $\mathsf{CL.Enc}(\gamma_i x)$. The hiding of $\mathsf{CL.Enc}(\gamma_i(x+e))$ is inherited from $\mathsf{CL.Enc}(\gamma_i x)$. We next detail the computation of $\mathsf{CL.Enc}(\gamma_i x)$ via our modified scaling operation $\odot$.

Modified scaling operation. To begin with, we first introduce the sampling of the fresh randomness r. r should be chosen to ensure that ${\mathfrak{g}_\mathfrak{q}}^r$ is (near-)uniform over $\langle \mathfrak{g}_\mathfrak{q} \rangle$, which guarantees that $\phi_{K'} = \phi_K^a {\mathfrak{g}_\mathfrak{q}}^r$ reveals negligible information about exponent a. For simplicity, assume $r \xleftarrow{\$} \mathbb{Z}_{s'}$ (strictly speaking s' is unknown and uniform sampling from $[0, s')$ is impossible. However, as detailed in Section 2.2, we can sample r from a distribution statistically close to uniform over $\mathbb{Z}_{s'}$, maintaining valid security arguments). Our realization of the scaling operation $\odot$ is:

$$(\phi_{K^*}, \overline{am^{K^*}}) := a \odot (\phi_K, \overline{m^K}) = (\phi_K^{a+qr}, \left(\overline{m^K}\right)^{a+qr}).$$

Note that, if a itself is uniform in $\mathbb{Z}_q$, we can instead sample a uniformly from $\mathbb{Z}_{qs'}$, omit sampling r, and replace $a + qr$ by a in the above computation. This applies directly to our scaling operation $\mathsf{CL.Enc}(\gamma_i x) = \gamma_i \odot \mathsf{CL.Enc}(x)$, where γ_i (analogous to a) is sampled as $\gamma_i \xleftarrow{\$} \mathbb{Z}_{qs'}$.

Security intuition for modified scaling. Since threshold decryption relies on a common encapsulation, our goal is for the encapsulation ϕ_{K^*} to hide $a = ((a+qr) \bmod q)$, rather than relying on the entire CL ciphertext to do so. Because $\phi_K \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ and $s' = \mathrm{ord}(\mathfrak{g}_\mathfrak{q})$, $\phi_{K^*} = \phi_K^{a+qr}$ reveals at most $(a+qr) \bmod s'$.

[4] Our protocol technically requires multiplying $\phi_{K_3}^{sk_i}$ by a random $\langle \mathfrak{f} \rangle$-element $\mathfrak{f}^{\alpha_i}$ conditioned on $\sum_{j\in\mathcal{T}} \alpha_j = 0$, but we omit this detail here for simplicity.

Moreover, as q is coprime with s', $(a + qr) \bmod s'$ is uniform over $\mathbb{Z}_{s'}$ and independent of $(a + qr) \bmod q$, thus leaking no information about $(a + qr) \bmod q$.
Simpler ZKP relations. Our modified $\odot$ operation yields cheaper ZKPs for the broadcast encapsulation because the proved relation is simpler:

- Standard $\odot$ operation: prove knowledge of a, r such that $\phi_{K'} = \phi_K^a {\mathfrak{g}_\mathfrak{q}}^r$ (witness size: two scalars).
- Modified $\odot$ operation: prove knowledge of $w = a + qr$ such that $\phi_{K^*} = \phi_K^w$ (witness size: one scalar).

Put Everything Together—Our Signing Protocol. For subtasks 1 and 3, we follow DKL+23's efficient implementation. Summarizing above, the protocol execution for a signer P_i (holding $\mathsf{CL.Enc}(x) = (\phi_K, \overline{x^K})$) proceeds as follows:

- Round 1: Sample and commit to $e_i, s_i \xleftarrow{\$} \mathbb{Z}_q$. Compute

$$(\phi_{K^{\gamma_i}}, \overline{\gamma_i x^{K^{\gamma_i}}}) := (\phi_K^{\gamma_i}, \left(\overline{x^K}\right)^{\gamma_i})$$

  for $\gamma_i \xleftarrow{\$} \mathbb{Z}_{qs'}$, and broadcast $\phi_{K^{\gamma_i}}$ alongside a ZKP for its validity. The common encapsulation ϕ_{K^γ} is computed as the product of all received $\phi_{K^{\gamma_i}}$.
- Round 2: Release (e_i, s_i); aggregate to obtain (e, s).
- Round 3: Compute $\overline{\gamma_i (x + e)^{K^{\gamma_i}}} := \overline{\gamma_i x^{K^{\gamma_i}}} \cdot \mathfrak{f}^{\gamma_i e}$. Then broadcast (pd_i, R_i) to derive (u, R), and output the signature $(A := \frac{R}{u}, e, s)$ if it is verified as valid.

We do not require a ZKP for pd_i because $\{pd_j\}_{j \in \mathcal{T}}$ already determines the MAC key $\gamma \bmod q$ (or yields $\perp$, which is trivially invalid). Hence, security follows directly from DKL+23. Implicitly, the ZKP for $\phi_{K^{\gamma_i}}$ must be a Zero-Knowledge Proof of Knowledge (ZKPoK) to support extraction of each γ_i (as in DKL+23 via MtA). The extracted γ_i captures the corrupted signer's contribution to the MAC key and is used to check whether the broadcast values (pd_i, R_i) are consistent with that contribution, which is crucial for the simulation.

Our Protocol vs. WMC24. The dominant cost in both our protocol and WMC24 is due to operations on class-group elements. We therefore provide a rough per-signer comparison focusing on (i) the number of broadcast group elements and (ii) the number of exponentiations in $\langle \mathfrak{g}_\mathfrak{q} \mathfrak{f} \rangle \backslash \langle \mathfrak{f} \rangle$, which highly reflects communication and computation cost, respectively. Our protocol requires each signer to broadcast only 2 group elements and to perform $\sim 2t$ exponentiations (for the DL-relation ZKPoK on $\phi_{K^{\gamma_i}}$). For the WMC24 protocol, it requires 9 broadcast group elements (2×3 for nonce generation and 3 for scaled ciphertext plus threshold decryption), and $\sim 29t$ exponentiations—specifically, $\sim 2 \times 9t$ for the nonce-generation ZKP, $\sim 6t$ for the ciphertext-scaling ZKP, and $\sim 5t$ for the threshold-decryption ZKP. Consequently, our protocol achieves better communication efficiency (2 vs. 9) and nearly an order-of-magnitude reduction in computational overhead ($\sim 2t$ vs. $\sim 29t$).

Remark 3. In WMC24, each signer must exponentiate all received pd_j by the respective Lagrange interpolation coefficients. This design enables the same pd_j to be reused across different signer sets, which is needed for the robust property in their paper. For a fixed signer set, each signer can pre-exponentiate their Lagrange coefficients directly onto the pd_j they broadcast. This reduces $1t$ exponentiations per threshold decryption instance, and saves $3t$ exponentiations across the entire WMC24 protocol.

1.4 Related Works

Other threshold BBS+. Faust et al. [19] introduced a threshold BBS+ scheme leveraging pseudorandom correlation generators and functions, but their construction operates in the preprocessing model—explicitly avoided in our work for reasons aligned with DKL+23: preprocessing consumes finite correlated states offline, risking reuse and requiring periodic replenishment.

Gennaro et al. [22] and Tang-Xue [28] designed threshold BBS+ protocols that, like DKL+23, employ MtA conversions to compute $u = \gamma(x + e)$ (subtask 2). Gennaro et al. instantiate their MtA using Paillier encryption, but require expensive zero-knowledge range proofs, incurring significant overhead. Tang-Xue (hereinafter referred to as TX25) implement MtA via CL encryption and achieve robust and IA properties by attaching ZKPs to each MtA instance. They further separate signing execution into an offline phase (message-independent) and an online phase (message-dependent), shifting all costly operations offline to optimize online efficiency. Since this separation coincides with preprocessing, we instead focus on *overall* performance. TX25 exhibits $\mathcal{O}(t^2)$ complexity from verifying $\mathcal{O}(t^2)$ MtA instances, resulting in slower execution than WMC24 for $t \geq 20$. Its bandwidth also grows linearly with t, exceeding WMC24 when $t \geq 10$.

Recent two-round threshold ECDSA. Recent threshold ECDSA protocols by DNP25 and LLZ+25 introduce novel secure multiplication techniques that save one round compared to previous approaches (e.g., computing u via MtA in DKL+23). Both protocols operate within the CL framework. We observe that our homomorphic scaling technique from Section 1.3 can be adapted to these protocols: parties broadcast a commitment component structurally similar to the first element $\phi_{K'}$ of the scaled CL ciphertext. By replacing this with an element of form ϕ_{K^*}, we achieve reductions in both communication and computation costs during their multiplication protocol. Additional discussion appears in Section 5.

These multiplication protocols are applicable to threshold BBS+/BBS signing with asymptotic complexity equivalent to our technique. While they outperform our method for one-time key generation by avoiding CL distributed key generation (DKG), our approach proves more efficient for signing executions. Section 5 provides a concrete comparison (measured in exponentiations over class-group elements) between our method and the optimized multiplication protocols from DNP25 and LLZ+25.

2 Preliminaries

Let λ and λ_d (where $\lambda \geq \lambda_d$) denote the computational and statistical security parameters, respectively. We write $\approx_c$ and $\approx_s$ for computational indistinguishability in λ and statistical indistinguishability in λ_d. Let n be the number of parties and t be the threshold size. For any positive integer s, let $[s]$ denote the set of integers within the range $[1, s]$. Boldface denotes vectors, e.g., a ℓ-dimensional vector $\mathbf{m} = (m_1, \ldots, m_\ell)$. Probabilistic polynomial time is abbreviated as PPT.

We consider static PPT adversaries that corrupts parties only at the onset of the protocol. Our security proof adopts the simulation-based security model [24], requiring that the real-world protocol execution be indistinguishable from an ideal execution with a trusted party computing the functionality. This guarantees that adversaries learn no information beyond the intended outputs. We assume ideal broadcast channels and authenticated, private point-to-point channels.

2.1 Bilinear Groups and BBS+ Signatures

A bilinear group (or pairing group) is described by a tuple $\mathcal{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, q, \mathsf{e})$ where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_t$ are cyclic groups of prime order q, with $G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ as respective generators. The efficient non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_t$ satisfies:

$$\forall x, y \in \mathbb{Z}_q,\ e(x \cdot G_1, y \cdot G_2) = xy \cdot e(G_1, G_2).$$

Given the group description $\mathcal{G}$ and the message vector length ℓ, the BBS+ scheme [1,6] works as follows:

- $\mathsf{Kgen}(\mathcal{G}, \ell, 1^\lambda) \to (x, X, \mathbf{H})$: let $x \xleftarrow{\$} \mathbb{Z}_q$ and $\mathbf{H} \xleftarrow{\$} \mathbb{G}_1^{\ell+1}$ and compute $X := x \cdot G_2 \in \mathbb{G}_2$.
- $\mathsf{Sign}((x, \mathbf{H}), \mathbf{m} \in \mathbb{Z}_q^\ell) \to (A, e, s)$: let $e, s \xleftarrow{\$} \mathbb{Z}_q$ and compute

$$A := \frac{G_1 + s \cdot H_1 + \sum_{i \in [\ell]} m_i \cdot H_{i+1}}{x + e}.$$

- $\mathsf{Verify}((X, \mathbf{H}), \mathbf{m} \in \mathbb{Z}_q^\ell, (A, e, s)) \to 0/1$: output 1 iff

$$\mathsf{e}(A, X + e \cdot G_2) = \mathsf{e}(G_1 + s \cdot H_1 + \sum_{i \in [\ell]} m_i \cdot H_{i+1}, G_2).$$

The ideal (t, n) threshold BBS+ functionality $\mathcal{F}_{\mathsf{BBS+}}$ is defined in Fig. 1. Our threshold BBS+ construction targets this functionality, which provides security with abort (consistent with [18]).

$\mathcal{F}_{\mathsf{BBS+}}$ is parameterized by the group description $\mathcal{G}$ and integers ℓ and λ, and works with n parties $P_1, \ldots, P_n$ as follows:

- **Keygen:** Upon receiving $(\mathsf{Kgen}, \mathrm{sid})$ from all parties, store $(x, X, \mathbf{H}) \leftarrow \mathsf{Kgen}(\mathcal{G}, \ell, 1^\lambda)$ in memory, then output $(\mathsf{vk}, \mathrm{sid}, (X, \mathbf{H}))$ to all parties.
- **Sign:** Upon receiving $(\mathsf{Sign}, \mathrm{sid}, \mathbf{m} \in \mathbb{Z}_q^\ell, \mathcal{T})$ from all signers (parties indexed by $\mathcal{T}$) where $|\mathcal{T}| \geq t$, if a record of the form $(x, X, \mathbf{H})$ exists in memory and sid is fresh, compute $(A, e, s) \leftarrow \mathsf{Sign}((x, \mathbf{H}), \mathbf{m})$ and output $(\mathsf{signature}, \mathrm{sid}, (A, e, s))$ to all signers; otherwise, ignore this command.

Fig. 1: The (t, n) threshold BBS+ functionality $\mathcal{F}_{\mathsf{BBS+}}$.

2.2 CL Framework and CL KEM

The CL framework [12,13] is defined over a multiplicative cyclic group $\mathfrak{G} = \langle \mathfrak{g}_\mathfrak{q} \mathfrak{f} \rangle$. Assign $\mathfrak{g}$ as $\mathfrak{g}_\mathfrak{q}\mathfrak{f}$, then $\mathfrak{G}$ is also generated by $\mathfrak{g}$. $\mathfrak{G}$ constitutes a subgroup of the class group $\hat{\mathfrak{G}}$ where $|\hat{\mathfrak{G}}| = \hat{s}q$ and $\gcd(\hat{s}, q) = 1$. Elements $\mathfrak{g}_\mathfrak{q}$ and $\mathfrak{f}$ in $\mathfrak{G}$ have respective orders s' and q satisfying $s' | \hat{s}$, implying $|\mathfrak{G}| = s'q$ and $\mathfrak{F} \cap \langle \mathfrak{g}_\mathfrak{q} \rangle = \{1\}$ where $\mathfrak{F} = \langle \mathfrak{f} \rangle$. There exists an efficient discrete logarithm algorithm $\mathsf{CL.Solve}$ for $\mathfrak{F}$ such that for any element $\mathfrak{f}^x \in \mathfrak{F}$,

$$x \equiv x' \bmod q \quad \text{where} \quad x' \leftarrow \mathsf{CL.Solve}(\mathfrak{f}^x).$$

The CL parameter set $pp_{cl} = (q, \overline{s}, \mathfrak{f}, \mathfrak{g}_\mathfrak{q}, \hat{\mathfrak{G}}, \mathfrak{F}; \rho)$ is generated by a probabilistic algorithm $\mathsf{CL.Gen}(1^\lambda, q)$, where ρ denotes the randomness used in $\mathsf{CL.Gen}$ and $\overline{s} \geq \hat{s}$ serves as an upper bound for $\hat{s}$ whose exact value remains unknown.

Define $\mathcal{B} := 2^{\lambda_d} q\overline{s}$ and $\mathcal{B}_q := 2^{\lambda_d}\overline{s}$. In the CL framework, randomness is sampled uniformly from the intervals $[0, \mathcal{B})$ and $[0, \mathcal{B}_q)$, inducing distributions over $\mathbb{Z}_{qs'}$ and $\mathbb{Z}_{s'}$ that are within statistical distance at most $2^{-\lambda_d}$ from uniform, respectively [10]. We denote by $\mathcal{D}$ (resp. $\mathcal{D}_q$) the uniform distribution over $[0, \mathcal{B})$ (resp. $[0, \mathcal{B}_q)$). For our security proof, we additionally use $\mathcal{D}'$, the uniform distribution over $[0, 2^\lambda q\overline{s})$, which induces distributions over $\mathbb{Z}_{qs'}$ and $\mathbb{Z}_{s'}$ that are within statistical distance at most $2^{-\lambda}$ from uniform.

The CL encryption scheme operates over the CL framework, but we do not introduce it; rather, as stated in Section 1.3, we devise a Key Encapsulation Mechanism (KEM) as CL KEM. Under the parameter set pp_{cl} and secret-public key pair (sk, pk) satisfying $pk = {\mathfrak{g}_\mathfrak{q}}^{sk}$,[5] CL KEM functions as follows:

- $\mathsf{Encap}(pk; r) \rightarrow (\phi_K, K)$: let $r \leftarrow \mathcal{D}_q$ and output $\phi_K := {\mathfrak{g}_\mathfrak{q}}^r$ and $K := pk^r$. The subscript of ϕ_K indicates that ϕ_K encapsulates K.
- $\mathsf{Decap}(sk, \phi_K) \rightarrow K$: output $K := \phi_K^{sk}$.

[5] In our setting, sk is jointly contributed by multiple parties rather than generated by a single party via an algorithm; we thus omit its generation process here.

- $\mathsf{Enc}(K, m) \to \overline{m^K}$: output $\overline{m^K} := K\mathfrak{f}^m$. The superscript in $\overline{m^K}$ denotes the ciphertext of m encrypted under key K.
- $\mathsf{Dec}(K, \overline{m^K}) \to m$: output $\mathsf{CL.Solve}(\frac{\overline{m^K}}{K})$.
  Naturally, $\mathsf{Enc}(1, m) = \overline{m^1}$ is defined as $\mathfrak{f}^m$.

We here emphasize again that CL KEM is not a secure KEM, but rather a reinterpretation of threshold CL encryption for better stating our distinction with prior works. Our fundamental security tools are the Hard Subgroup Membership (HSM) assumption and smoothness lemma. HSM assumption states that a uniformly distributed element in $\mathfrak{G}$ is indistinguishable from one in $\langle \mathfrak{g}_\mathfrak{q} \rangle$. The smoothness lemma leverages the algebraic structure of groups $\mathfrak{G}$, $\mathfrak{F}$, and $\langle \mathfrak{g}_\mathfrak{q} \rangle$: for any $x \in \mathfrak{G} \setminus \langle \mathfrak{g}_\mathfrak{q} \rangle$ and ${\mathfrak{g}_\mathfrak{q}}^d$ where d is (near-)uniform over $\mathbb{Z}_{qs'}$, decomposing x^d as ${\mathfrak{g}_\mathfrak{q}}^a\mathfrak{f}^b$ yields the exponent b distributed (near-)uniformly over $\mathbb{Z}_q$.

Definition 1 (HSM assumption). *Let λ be the security parameter, $q \geq 2^\lambda$ be a prime. For any PPT algorithm $\mathcal{A}$, the advantage defined below*

$$\mathsf{Adv}_{\mathcal{A}}^{HSM}(\lambda, q) := |2 \cdot \Pr[b = b' : pp_{cl} \leftarrow \mathsf{CL.Gen}(1^\lambda, q),$$
$$x \leftarrow \mathcal{D}', Z_0 := \mathfrak{g}^x, Z_1 := {\mathfrak{g}_\mathfrak{q}}^x, b \xleftarrow{\$} \{0,1\}, b' \leftarrow \mathcal{A}(pp_{cl}, Z_b)] - 1|$$

is negligible in λ.

Lemma 1 (Smoothness). *For any $x \in \mathfrak{G} \setminus \langle \mathfrak{g}_\mathfrak{q} \rangle$, $\pi \xleftarrow{\$} \mathfrak{F}$, and $d \leftarrow \mathcal{D}$, the two distributions $(x, {\mathfrak{g}_\mathfrak{q}}^d, \pi x^d)$ and $(x, {\mathfrak{g}_\mathfrak{q}}^d, x^d)$ are statistically indistinguishable.*

2.3 Zero-Knowledge Proofs

Let $\mathcal{R}$ denote a relation with corresponding NP language $\mathcal{L}$. A zero-knowledge proof (ZKP) allows a prover in possession of a statement-witness pair $(x, w) \in \mathcal{R}$ to (interactively) convince a verifier that $x \in \mathcal{L}$ without revealing w. In our application of ZKPs, we apply the Fiat-Shamir heuristic [20] to convert the interactive ZKP into a non-interactive protocol $\Pi_\mathcal{R} = (\Pi_\mathcal{R}.\mathsf{Prv}, \Pi_\mathcal{R}.\mathsf{Vrf})$:

- $\Pi_\mathcal{R}.\mathsf{Prv}(x, w) \to \pi$: this algorithm is executed by the prover and outputs a proof π.
- $\Pi_\mathcal{R}.\mathsf{Vrf}(x, \pi) \to \{0, 1\}$: this algorithm is executed by the verifier and outputs 1 iff π is a valid proof for $x \in \mathcal{L}$.

Full requirements of non-interactive ZKPs $\Pi_\mathcal{R}$ can be found in Appendix B.1.

When witness extraction is required, we employ zero-knowledge proofs of knowledge (ZKPoKs), formally modeled by the functionality $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}}$ in Fig. 2.

Our threshold BBS+ construction (Figs. 6, 7, and 8) employs ZKPs for $(\mathcal{R}_{\text{dlc}}, \mathcal{R}_{\mathsf{enc}})$ and ZKPoKs for $(\mathcal{R}_{\text{dlc}}, \mathcal{R}_{\text{dls}})$:

- *Discrete logarithm in class group.*

$$\mathcal{R}_{\text{dlc}} := \{((D, h), w) : D = h^w\}$$

  where $h \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ and $w \in \mathbb{Z}$.

$\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}}$ works with parties $P_1, \ldots, P_n$ as follows:
- On receiving $(\mathsf{prove}, \mathrm{sid}, \mathcal{T}, x, w)$ from party P_i for $i \in [n]$ and sid is never used by P_i, send $(\mathsf{proof}, \mathrm{sid}, \mathcal{T}, i, x, b)$ to all parties indexed by $\mathcal{T}$ where $b \in \{0, 1\}$ and $b = 1$ iff $(x, w) \in \mathcal{R}$.

Fig. 2: The zero-knowledge functionality $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}}$.

- *Consistency of the use of the message between CL encapsulation-ciphertext pair and the public key exponent.*

$$\mathcal{R}_{\mathsf{enc}} := \left\{ \begin{array}{l} ((pk, \phi_K, \overline{m^K}, M), (m, r)) : \\ \phi_K = {\mathfrak{g}_\mathfrak{q}}^r \wedge \overline{m^K} = pk^r \mathfrak{f}^m \wedge M = mG_2 \end{array} \right\}$$

  where $pk \in \langle \mathfrak{g}_\mathfrak{q} \rangle$, $m \in \mathbb{Z}_q$ and $r \in \mathcal{D}_q$.
- *Special discrete logarithm in class group.*

$$\mathcal{R}_{\mathrm{dls}} := \{((c, C), (w_1, w_2)) : C = c^{w_1 \cdot 2^{-w_2}}\}$$

  where $c \in \langle \mathfrak{g}_\mathfrak{q} \rangle$, $w_1 \in \mathcal{D}$ and $w_2 \in \mathbb{Z}$.

See Appendix B.2 for implementation details. One important technical detail is that $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}}$ is employed to extract γ_i from $\phi_{K^{\gamma_i}} = \phi_K^{\gamma_i}$ during the signing phase (Fig. 8), where ϕ_K is an encapsulation. The extraction technique follows [11], which guarantees the extraction of γ_i provided that ϕ_K is unbiased and uniformly distributed in $\langle \mathfrak{g}_\mathfrak{q} \rangle$. To satisfy this condition, we construct ϕ_K using a commit-and-release paradigm.

Remark 4. It is worth noting that the witness extraction in our scheme inherently relies on rewinding techniques. Since rewinding is generally incompatible with the Universal Composability (UC) framework [9]—which imposes significantly stronger security guarantees than standard simulation-based models—our protocol is proven secure under standard simulation-based security rather than achieving full UC security.

2.4 Publicly Verifiable Secret Sharing

Publicly verifiable secret sharing (PVSS) enables a dealer to distribute a secret among a set of parties while allowing reconstruction by any subset meeting the threshold. The public verifiability property further ensures that each party's share can be publicly validated. In our protocol, we employ this technique to distribute each party's contribution v_i to the implicitly-existed CL secret key sk. Note that we treat v_i as an integer rather than an element of $\mathbb{Z}_{s'}$ since the modulus s' is unknown. To implement PVSS within the CL framework, we adopt the method by Braun *et al.* [4, Section 4], which allows a party to share a secret $s \in [0, \mathcal{B}_q)$ as follows:

0. Set Δ as the factorial $n!$ and $\ell_0 = \lceil \log(\mathcal{B}_q) \rceil + 1 + \lceil \log_2(t(t-1)n^n) \rceil$.
1. Sample coefficients $\{a_j\}_{j=1}^{t-1} \xleftarrow{\$} [0, 2^{\ell_0+\lambda_d})$. For each $i \in [n]$, compute $y_i := h(i)$ where $h(x) = \Delta s + \sum_{j=1}^{t-1} a_j x^j$. Meanwhile, compute $C_0 := {\mathfrak{g}_\mathfrak{q}}^s$ and $C_j := {\mathfrak{g}_\mathfrak{q}}^{\Delta a_j}$ for $j \in [t-1]$, and generate a ZKPoK for C_0 and ZKPs for $\{C_j\}_{j=1}^{t-1}$ to attest their correctness.
2. Send y_i to party P_i as his share and broadcast $\{C_j\}_{j=0}^{t-1}$ along with the proofs.
3. Each party P_i verifies the proofs and checks the share y_i by validating

$$\mathsf{SScheck}_i(\{C_j\}_{j=0}^{t-1}, y_i) \to 0/1\text{: output 1 iff } {\mathfrak{g}_\mathfrak{q}}^{\Delta \cdot y_i} = C_0^{\Delta^2} \cdot \prod_{j=1}^{t-1} {C_j}^{i^k}.$$

The PVSS scheme hides s via the privacy lemma (Lemma 2), establishing that distributions of corrupted parties' shares are 'almost" identical across different secrets. The Feldman simulation (Lemma 3) guarantees perfect simulability of the corrupted parties' view on $\{C_j\}_{j=1}^{t-1}$ when given C_0 and their shares.

Lemma 2 (Privacy, from [4]). *For any set of corrupted parties indexed by* $\mathbf{P}^*$ *with* $|\mathbf{P}^*| < t$ *and any two* $\alpha, \alpha' \in \mathcal{D}_q$ *and any two independent uniform randomness* $\mathbf{a}, \mathbf{a}' \in [0, 2^{\ell_0+\lambda_d})^{t-1}$*, the distributions* $\{y_i | y_i = \Delta\alpha + \sum_{j=1}^{t-1} a_j i^j\}_{i \in \mathbf{P}^*}$ *and* $\{y'_i | y'_i = \Delta\alpha' + \sum_{j=1}^{t-1} a'_j i^j\}_{i \in \mathbf{P}^*}$ *are statistically indistinguishable.*

Lemma 3 (Feldman Simulation, from [4]). *Given* $C_0 \in G^q$ *and* (i_j, y_{i_j}) *with* $i_j \le N$ *for every* $j \in [t-1]$*, we can efficiently find* $C_1, \ldots, C_{t-1} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ *such that* $\mathsf{SScheck}_{i_j}(\{C_k\}_{k=0}^{t-1}, y_{i,j}) = 1$ *for every* $j \in [t-1]$*. Moreover, the values of* $(C_1, \ldots, C_{t-1})$ *have, conditioned on* C_0 *and* $\{(i_j, y_{i_j})\}_{j \in [t-1]}$*, the same distribution as produced of real broadcast* $(C_1, \ldots, C_{t-1})$ *when sharing* s *for* ${\mathfrak{g}_\mathfrak{q}}^s = C_0$*.*

From [4, Lemma 6], any t valid shares y_i (extracted via $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}}$ in our protocol) enable reconstructing s through Lagrange interpolation, consistent with C_0's witness even when coefficients in $h(x)$ are maliciously generated (that is, $a_j \notin [0, 2^{\ell_0+\lambda_d})$ or $s \notin \mathcal{D}_q$). The reconstruction is essentially Lagrange interpolation:

$$s = \frac{1}{\Delta^2} \sum_{i \in \mathcal{T}} (\Delta L_i^{\mathcal{T}}(0)) y_i \tag{1}$$

where $\mathcal{T} \subseteq [n]$ with $|\mathcal{T}| = t$, and $L_i^{\mathcal{T}}(k) := \prod_{j \in \mathcal{T} \setminus \{i\}} \frac{j-k}{j-i}$ denotes the Lagrange coefficient. Although $L_i^{\mathcal{T}}(k)$ may be fractional, $\Delta L_i^{\mathcal{T}}(k)$ is integer-valued for any $\mathcal{T} \subseteq [0, \ldots, n]$ and $k \in \mathbb{Z}$.

2.5 Other Functionalities

The distributed setup functionality $\mathcal{F}_{\mathsf{cl}}$ for parameters pp_{cl} is specified in Fig. 3. Its realization employs the commitment functionality $\mathcal{F}_{\mathsf{COM}}$ (defined in Fig. 4) within a commit-and-release procedure to sample randomness ρ, where $\mathcal{F}_{\mathsf{COM}}$ is

instantiated via a cryptographic hash (e.g., SHA256 for the 128-bit security). Then, each party can run $\mathsf{CL.Gen}(\cdot)$ locally with randomness ρ to obtain pp_{cl}.

Additionally, our signing protocol employs a zero-sharing functionality $\mathcal{F}_{\text{zero}}$ (Fig. 5), whose output re-randomizes the partial decryptions. Our realization follows DKL+23 and proceeds in two phases: a one-time setup phase and a sampling phase that can be executed multiple times. Our realization follows DKL+23 and proceeds in two phases:

- **One-time setup phase:** Each pair of parties (P_i, P_j) jointly generates a seed $\beta_{i,j}$ via a commit-and-release protocol (ensuring unbiasedness). Concurrently, each party P_i initializes counters $\{\mathsf{cnt}_{i,j}\}_{j \neq i}$ to zero.
- **Sampling phase:** To sample zero shares among parties indexed by $\mathcal{T}$, each P_i for $i \in \mathcal{T}$ locally computes

$$\alpha_i = \sum_{i<j \in \mathcal{T}} H(\mathsf{cnt}_{i,j}, \beta_{i,j}) - \sum_{i>j \in \mathcal{T}} H(\mathsf{cnt}_{i,j}, \beta_{j,i}),$$

  and increments $\{\mathsf{cnt}_{i,j}\}_{j \in \mathcal{T}}$. Here, $H(\cdot)$ denotes a cryptographic hash function with output mapped to $\mathbb{Z}_q$, modeled as a random oracle to ensure security.

3 Our Threshold BBS+ Construction

In this section, we present our threshold BBS+ signature design. Our construction realizing $\mathcal{F}_{\mathsf{BBS+}}$ appears in Section 3.1, with its security analysis detailed in Section 3.2.

3.1 Main Construction

We now describe the overall protocol for our main construction. The parties execute the protocol with public parameters $(\mathcal{G}, \lambda, \lambda_d, \ell)$, which fixes the bilinear group and security parameters.

Key generation. This protocol executes only once, and we divide its 5-round execution into two parts. Part I handles distributed key generation (DKG) for CL KEM, as depicted in Fig. 6. Part II establishes the BBS+ verifying keys, a public encapsulation ϕ_K and symmetric ciphertext $\overline{x^K}$ of the signing key encrypted under the decapsulated key K, detailed in Fig. 7. For clarity, we also describe the execution in the text below. Note that any failed verification or check causes the protocol to abort, which is omitted in the following description by default.

$\mathcal{F}_{\mathsf{cl}}$ works with parties $P_1, \ldots, P_n$ as follows:

- On receiving $(\mathsf{clgen}, 1^\lambda, q)$ from all parties, compute $(q, \overline{s}, \mathfrak{f}, \mathfrak{g}_\mathfrak{q}, \hat{\mathfrak{G}}, \mathfrak{F}; \rho) \leftarrow \mathsf{CL.Gen}(1^\lambda, q)$ and output $pp_{cl} := (q, \overline{s}, \mathfrak{f}, \mathfrak{g}_\mathfrak{q}, \hat{\mathfrak{G}}, \mathfrak{F}; \rho)$ to all parties.

Fig. 3: The distributed setup functionality $\mathcal{F}_{\mathsf{cl}}$ for pp_{cl}.

$\mathcal{F}_{\mathsf{COM}}$ works with parties $P_1, \ldots, P_n$ as follows:
- On receiving $(\mathsf{commit}, \mathrm{sid}, \mathcal{T}, x)$ from party P_i for $i \in [n]$, record $(\mathrm{sid}, \mathcal{T}, i, x)$ and send $(\mathsf{commit}, \mathrm{sid}, \mathcal{T}, i)$ to all parties indexed by $\mathcal{T}$. If some $(\mathsf{commit}, \mathrm{sid}, \mathcal{T}, \cdot)$ has already been recorded, ignore the message.
- On receiving $(\mathsf{decommit}, \mathrm{sid})$ from party P_i for $i \in [n]$, if some $(\mathrm{sid}, \mathcal{T}, i, x)$ is recorded then send $(\mathsf{decommit}, \mathrm{sid}, \mathcal{T}, i, x)$ to all parties indexed by $\mathcal{T}$.

Fig. 4: The commitment functionality $\mathcal{F}_{\mathsf{COM}}$.

$\mathcal{F}_{\mathsf{zero}}$ is parameterized by q and works with parties $P_1, \ldots, P_n$ as follows:
- On receiving $(\mathsf{sample}, \mathrm{sid}, \mathcal{T})$ from some party P_i for $i \in \mathcal{T}$ where $\mathcal{T} \subseteq [n]$ and sid is fresh, sample $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{Z}_q^{|\mathcal{T}|}$ conditioned on $\sum_{j \in \mathcal{T}} \alpha_j = 0$, and send $(\mathsf{zero-share}, \mathrm{sid}, \alpha_i)$ to P_i. Upon receiving $(\mathsf{sample}, \mathrm{sid}, \mathcal{T})$ from any other P_j for $j \in \mathcal{T} \setminus \{i\}$, send $(\mathsf{zero-share}, \mathrm{sid}, \alpha_j)$ to P_i.

Fig. 5: The zero-sharing functionality $\mathcal{F}_{\mathsf{zero}}$.

(i) Distributed setup of the CL framework:
All parties jointly invokes $\mathcal{F}_{\mathsf{cl}}$ to establish the public parameter pp_{cl} for the CL framework, which takes 2 rounds.

(ii) DKG of CL KEM via PVSS
In round 3, each party P_i generates a secret $v_i \leftarrow \mathcal{D}_q$ and employs the PVSS scheme from Section 2.4 to share v_i. This results in $y_{i,j}$ being released to P_j as his share and $C_{i,0} := {\mathfrak{g}_\mathfrak{q}}^{v_i}$ being published publicly. Then, P_i stores the CL public key $pk := \prod_{j \in [n]} C_{j,0}$ and value $sk_i := \sum_{j \in [n]} y_{j,i}$. Since $pk := {\mathfrak{g}_\mathfrak{q}}^{sk}$ where $sk := \sum_{j \in [n]} v_j$ represents the implied CL secret key, each v_j constitutes P_j's contribution to sk, making sk_i a valid share of sk. In round 4, each P_i computes D_i from public information, which should equal $\mathfrak{g}_{\mathfrak{q}\Delta}^{sk_i} = \mathfrak{g}_\mathfrak{q}^{\Delta sk_i}$ ($\mathfrak{g}_{\mathfrak{q}\Delta}$ is set as $\mathfrak{g}_\mathfrak{q}^{\Delta}$) by Lagrange interpolation, and proves his knowledge of sk_i relative to D_i via $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}}$.

(iii) Setup $(X, \mathbf{H})$, encrypt x and release the key encapsulation:
In round 4 (it is executed concurrently with the aforementioned round 4 procedures as all required information is available by the end of round 3), P_i generates an encapsulation-key pair (ϕ_{K_i}, K_i), samples $x_i \xleftarrow{\$} \mathbb{Z}_q$ and $(X_i = x_i \cdot G_2, \mathbf{H}_i \xleftarrow{\$} \mathbb{G}_1^{\ell+1})$ as his contribution to the BBS+ signing-verifying key pair, and encrypts x_i as $\overline{x_i^{K_i}}$. P_i then broadcasts a commitment to $(\phi_{K_i}, \overline{x_i^{K_i}}, \mathbf{H}_i, X_i)$. In round 5, P_i opens this commitment and provides a ZKP validating $(\phi_{K_i}, \overline{x_i^{K_i}}, X_i)$. Finally, P_i obtains the BBS+ verifying key $(X = \sum_{j \in [n]} X_j, \mathbf{H} = \sum_{j \in [n]} \mathbf{H}_j)$, a public encapsulation

DKG of CL KEM under the session id sid.

Every party P_i, where $i \in [n]$ **Parties** $\{P_j\}_{j\in[n]\setminus\{i\}}$

(i) **Distributed setup of the CL framework**

Rounds 1 and 2: $\xrightarrow{(\mathsf{clgen},1^\lambda,q)} \mathcal{F}_{\mathsf{cl}} \xrightarrow{(q,\overline{s},\mathfrak{f},\mathfrak{g}_\mathfrak{q},\hat{\mathfrak{G}},\mathfrak{F};\rho)}$

store $(q, \overline{s}, \mathfrak{f}, \mathfrak{g}_\mathfrak{q}, \hat{\mathfrak{G}}, \mathfrak{F}; \rho)$ as pp_{cl}

(ii) **DKG of CL KEM via PVSS**

Round 3:

$v_i \leftarrow \mathcal{D}_q,\ C_{i,0} := {\mathfrak{g}_\mathfrak{q}}^{v_i},$

$\Delta := n!,\ \mathfrak{g}_{\mathfrak{q}\Delta} := {\mathfrak{g}_\mathfrak{q}}^{\Delta} \xrightarrow{(\mathsf{prove},\mathsf{sid},[n],(C_{i,0},\mathfrak{g}_\mathfrak{q}),v_i)} \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}} \xrightarrow{(\mathsf{proof},\ldots,b_{i,0})}$ If $b_{i,0} = 0$, **abort**

$\ell_0 := \lceil \log(B_q) \rceil + 1 + \lceil \log(t(t-1)n^n) \rceil$

$\mathbf{a_i} := \{a_{i,k}\}_{k\in[t-1]} \xleftarrow{\$} [0, 2^{\ell_0+\lambda_d})^{t-1}$

For $k \in [t-1]$:

$C_{i,k} := \mathfrak{g}_{\mathfrak{q}\Delta}^{a_{i,k}}$

$\pi_{i,k} \leftarrow \Pi_{\mathcal{R}_{\mathrm{dlc}}}.\mathsf{Prv}((C_{i,k}, \mathfrak{g}_{\mathfrak{q}\Delta}), a_{i,k}) \xrightarrow[\text{Broadcast to all parties}]{(\mathsf{zk\text{-}verify0},\mathsf{sid},i,\{C_{i,k}\}_{k=1}^{t-1},\{\pi_{i,k}\}_{k=1}^{t-1})}$

If $\exists k$ s.t. $\Pi_{\mathcal{R}_{\mathrm{dlc}}}.\mathsf{Vrf}((C_{i,k}, \mathfrak{g}_{\mathfrak{q}\Delta}), \pi_{i,k}) = 0$, **abort**

For $j \in [n] : y_{i,j} := v_i\Delta + \sum_{k=1}^{t-1} a_{i,k} j^k \xrightarrow[\text{Privately send to party } P_j \text{ for every } j\neq i]{(\mathsf{share},\mathsf{sid},i,y_{i,j})}$

If $\mathrm{SScheck}_j(\{C_{i,k}\}_{k=0}^{t-1}, y_{i,j}) = 0$, **abort**

Store $pk := \prod_{j\in[n]} C_{j,0}$ and $sk_i := \sum_{j\in[n]} y_{j,i}$

Round 4:

For $k \in [0, \ldots, t-1]$: $C_k := \prod_{j\in[n]} C_{j,k}$

$D_i := C_0^{\Delta^2} \cdot \prod_{k=1}^{t-1} {C_k}^{i^k} \xrightarrow{(\mathsf{prove},\mathsf{sid},[n],(D_i,\mathfrak{g}_{\mathfrak{q}\Delta}),sk_i)} \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}} \xrightarrow{(\mathsf{proof},\ldots,b_{i,1})}$

If $C_0^{\Delta^2} \prod_{k=1}^{t-1} {C_k}^{i^k} \neq D_i$ or $b_{i,1} = 0$, **abort**

Fig. 6: Threshold BBS+ key generation subprotocol part I.

$\phi_K := \prod_{j\in[n]} \phi_{K_j}$, and ciphertext $\overline{x^K} := \prod_{j\in[n]} \overline{x_j^{K_j}}$. The unbiasability of these values is ensured by the commit-and-release procedure. By homomorphic properties, $\overline{x^K}$ encrypts the implied signing key $x = \sum_{j\in[n]} x_j$ under $K = \prod_{j\in[n]} K_j$, which decapsulates ϕ_K.

Remark 5. Our DKG protocol of CL KEM (step (ii)) is adapted from [4, Fig. 4] by additionally appending a ZKPoK for knowledge of sk_i (implemented via $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}}$

Setup for the BBS+ verifying key and related components under the session id sid.

Every party P_i, where $i \in [n]$ **Parties** $\{P_j\}_{j\in[n]\setminus\{i\}}$

(iii) **Setup $(\boldsymbol{X}, \mathbf{H})$, encrypt x and release the key encapsulation**

Round 4:

$(\phi_{K_i}, K_i) \leftarrow \mathsf{Encap}(pk; r_i),\ x_i \xleftarrow{\$} \mathbb{Z}_q,\ \overline{x_i^{K_i}} \leftarrow \mathsf{Enc}(K_i, x_i)$

$X_i := x_i \cdot G_2,\ \mathbf{H}_i \xleftarrow{\$} \mathbb{G}_1^{\ell+1} \xrightarrow{(\mathsf{commit},\mathrm{sid},[n],(\phi_{K_i},\overline{x_i^{K_i}},\mathbf{H}_i,X_i))} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{commit},\ldots)}$

Round 5: $\xrightarrow{(\mathsf{decommit},\mathrm{sid})} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{decommit},\ldots,(\phi_{K_i},\overline{x_i^{K_i}},\mathbf{H}_i,X_i))}$

$\pi_i \leftarrow \Pi_{\mathcal{R}_{\mathsf{enc}}}.\mathsf{Prv}((pk, \phi_{K_i}, \overline{x_i^{K_i}}, X_i), (x_i, r_i))$

$\xrightarrow[\text{Broadcast to all parties}]{(\mathsf{zk\text{-}verify1},\mathrm{sid},i,\pi_i)}$

If $\Pi_{\mathcal{R}_{\mathsf{enc}}}.\mathsf{Vrf}((pk, \phi_{K_i}, \overline{x_i^{K_i}}, X_i), \pi_i) = 0$, **abort**

Store $X := \sum_{j\in[n]} X_j,\ \mathbf{H} := \sum_{j\in[n]} \mathbf{H}_j,$

and $\phi_K = \prod_{j\in[n]} \phi_{K_j}, \overline{x^K} \leftarrow \prod_{j\in[n]} \overline{x_j^{K_j}}$

Output $(\mathsf{vk}, \mathrm{sid}, (X, \mathbf{H}))$

Fig. 7: Threshold BBS+ key generation protocol part II.

in round 4) to enable extraction of sk_i. Whether this ZKPoK is necessary depends on the setting: in the honest-majority model of [4], where $\geq t$ parties are honest, it is redundant because sk_i can be extracted directly via Lagrange interpolation from honest shares; in our dishonest-majority model, with $< t$ honest parties, such extraction is infeasible, rendering the ZKPoK essential.

Signing. This 3-round protocol runs after the key generation phase completes, as depicted in Fig. 8. The protocol executes multiple times, each time taking as input a message $\mathbf{m} \in \mathbb{Z}_q^\ell$ and a signer index set $\mathcal{T} \subseteq [n]$ with $|\mathcal{T}| \geq t$. For clarity, we also describe the execution in the text below. Note that any failed verification or check causes the protocol to abort, which is omitted in the following description by default.

(iv) Commit (e_i, s_i), encrypt $\gamma_i x$ and construct a common encapsulation:
Each signer P_i generates and commits $e_i, s_i \xleftarrow{\$} \mathbb{Z}_q$ as his contribution to the signing nonces (e, s). Meanwhile, P_i samples the scaling factor $\gamma_i \leftarrow \mathcal{D}$, and computes the scaled encapsulation $\phi_{K^{\gamma_i}} := \phi_K^{\gamma_i}$ and ciphertext $\overline{\gamma_i x^{K^{\gamma_i}}} \leftarrow \left(\overline{x^K}\right)^{\gamma_i}$, then releases and validates $\phi_{K^{\gamma_i}}$ via $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}}$, which also

Threshold signing by parties indexed by $\mathcal{T}$ on message $\mathbf{m} \in \mathbb{Z}_q^\ell$ and the session id sid.

. . . . . .

Every party P_i, where $i \in \mathcal{T}$ **Parties** $\{P_j\}_{j\in\mathcal{T}\setminus\{i\}}$

. . . . . (iv) **Commit (e_i, s_i), encrypt $\gamma_i x$ and construct a common encapsulation** . . . . .

Round 1:

$(e_i, s_i) \xleftarrow{\$} \mathbb{Z}_q^2$ $\xrightarrow{(\mathsf{commit},\mathrm{sid},\mathcal{T},(e_i,s_i))} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{commit},\ldots)}$

$\gamma_i \leftarrow \mathcal{D},\ \phi_{K^{\gamma_i}} := \phi_K^{\gamma_i},\ \overline{\gamma_i x^{K^{\gamma_i}}} \leftarrow \left(\overline{x^K}\right)^{\gamma_i}$ $\xrightarrow{(\mathsf{prove},\mathrm{sid},\mathcal{T},(\phi_{\gamma_i},\phi),(\gamma_i,0))} \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}} \xrightarrow{(\mathsf{proof},\ldots,b_{\mathrm{sid}})}$

$\phi_{K^\gamma} := \prod_{j\in\mathcal{T}} \phi_{K^{\gamma_j}}$ If $b_{\mathrm{sid}} = 0$, **abort**

. . . . . (v) **Open (e_i, s_i), threshold-decrypt $\gamma(x+e)$ and derive the signature** . . . . . .

Round 2: $\xrightarrow{(\mathsf{decommit},\mathrm{sid})} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{decommit},\ldots,(e_i,s_i))}$

$e := \sum_{j\in\mathcal{T}} e_j,\ s := \sum_{j\in\mathcal{T}} s_j$

Round 3:

$B := G_1 + s \cdot H_1 + \sum_{k\in[\ell]} m_k \cdot H_{k+1},\ R_i = \gamma_i \cdot B,$

$\overline{\gamma_i(x+e)^{K^{\gamma_i}}} \leftarrow \overline{\gamma_i x^{K^{\gamma_i}}} \cdot \overline{\gamma_i e^1}$ (Note that $\overline{\gamma_i e^1} = \mathfrak{f}^{\gamma_i e}$),

Input $(\mathsf{sample}, \mathrm{sid}, \mathcal{T})$ to $\mathcal{F}_{\mathsf{zero}}$ and obtain α_i

$sk_i' := \Delta L_i^{\mathcal{T}}(0) \cdot sk_i,\ pd_i := \dfrac{\left(\overline{\gamma_i(x+e)^{K^{\gamma_i}}}\right)^{\Delta^2}}{\phi_{K^\gamma}^{sk_i'} \cdot \mathfrak{f}^{\alpha_i}}$ $\xrightarrow[\text{Broadcast to signing parties indexed by } \mathcal{T}]{(\mathsf{part\text{-}sig},\mathrm{sid},i,R_i,pd_i)}$

$u' \leftarrow \mathsf{CL.Solve}(\prod_{j\in\mathcal{T}} pd_j)$: If $u' = \perp$, **abort**

Compute $u := u' \cdot \Delta^{-2} \bmod q$ and $A := \dfrac{\sum_{j\in\mathcal{T}} R_j}{u}$

If $\mathsf{e}(A, X + e \cdot G_2) \neq \mathsf{e}(B, G_2)$, **abort**

Otherwise, **output** $(\mathsf{signature}, \mathrm{sid}, (A, e, s))$

Fig. 8: Threshold BBS+ signing protocol.

proves his knowledge of γ_i. Lastly, P_i concludes $\phi_{K^\gamma} = \prod_{j\in\mathcal{T}} \phi_{K^{\gamma_i}}$, where $\gamma = \sum_{j\in\mathcal{T}} \gamma_j$. The entire process completes in 1 round.

(v) Open (e_i, s_i), threshold-decrypt $\gamma(x+e)$ and derive the signature:
In round 2, each signer P_i opens the commitment to (e_i, s_i) and computes $e := \sum_{j\in\mathcal{T}} e_j$ and $s := \sum_{j\in\mathcal{T}} s_j$, with the unbiasability of (e, s) ensured by the commit-and-release procedure. In round 3, P_i computes B using public information and set $R_i := \gamma_i \cdot B$, noting that $\left(\frac{B}{x+e} = \frac{\sum_{j\in\mathcal{T}} R_j}{\gamma(x+e)},\ e,\ s\right)$ constitutes the expected signature. Concurrently, P_i computes ciphertext $\overline{\gamma_i(x+e)^{K^{\gamma_i}}}$ via homomorphic operations, observing that $\prod_{j\in\mathcal{T}} \overline{\gamma_j(x+e)^{K^{\gamma_j}}}$ forms a ciphertext of $\gamma(x+e)$ under key K^γ. To obtain $\gamma(x+e)$, P_i com-

putes partial decryption:

$$pd_i := \frac{\left(\overline{\gamma_i (x+e)^{K^{\gamma_i}}}\right)^{\Delta^2}}{\phi_{K^\gamma}^{sk'_i} \cdot \mathfrak{f}^{\alpha_i}},$$

where α_i denotes P_i's zero-share and $sk'_i := \Delta L_i^{\mathcal{T}}(0) \cdot sk_i$ serves as a valid sk share because $\sum_{j\in\mathcal{T}} sk'_j = \Delta^2 \cdot sk$ by equation 1, yielding

$$\prod_{j\in\mathcal{T}} pd_j = \frac{\overline{\Delta^2 \gamma (x+e)^{K^{\Delta^2\gamma}}}}{(\phi_{K^\gamma}^{sk})^{\Delta^2} \cdot \mathfrak{f}^0} = \mathfrak{f}^{\Delta^2\gamma(x+e)}.$$

Finally, P_i broadcasts R_i and pd_i, computes $u \leftarrow \Delta^{-2} \cdot \mathsf{CL.Solve}(\prod_{j\in\mathcal{T}} pd_j)$ under modulo-q arithmetic (which should equal $\gamma(x+e)$), and then derives and verifies the signature $\left(A := \frac{\sum_{j\in\mathcal{T}} R_j}{u},\ e,\ s\right)$.

3.2 Security Analysis

Our security theorem is formalized as:

Theorem 1. *Let the system parameters be $(\lambda, \lambda_d, \mathcal{G})$, and let $(\Pi_{\mathcal{R}_{dlc}}, \Pi_{\mathcal{R}_{\mathsf{enc}}})$ be ZKP protocols. Assuming the HSM assumption (Definition 1) holds, our protocols (depicted in Figs 6, 7 and 8) securely realize $\mathcal{F}_{\mathsf{BBS+}}$ with abort in the $(\mathcal{F}_{\mathsf{cl}}, \mathcal{F}_{\mathsf{COM}}, \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{dlc}}, \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{dls}})$-hybrid model against static malicious adversaries.*

Before presenting the full proof, we briefly sketch our proof technique. Since the high-level idea for securing our signing protocol is covered in Section 1.3, we focus exclusively on the key generation protocol's security, where the central challenge is ensuring that $(\phi_K, \overline{x^K})$ conceals the signing key x:

- Given $(\phi_K = \prod_{j\in[n]} \phi_{K_j}, \overline{x^K} = \prod_{j\in[n]} \overline{x_j^{K_j}})$ and the independent generation of $(\phi_{K_i}, \overline{x_i^{K_i}})$ for each honest index i, we can reduce the concealment of x in $(\phi_K, \overline{x^K})$ to that of x_k in $(\phi_{K_k}, \overline{x_k^{K_k}})$ for some fixed honest index k. We write $\overline{x_k^{K_k}}$ as $\phi_{K_k}^{\delta_k} \cdot \phi_{K_k}^{v_k} \mathfrak{f}^{x_k}$, where $\delta_k = sk - v_k$ is extractable in simulation.
- Change 1: sample v_k from $\mathcal{D}$ instead of $\mathcal{D}_q$. Using the privacy lemma and Feldman simulation, we simulate corrupted parties' shares and the values $\{C_{k,j}\}_{j=1}^{t-1}$, and, via Lagrange interpolation, simulate honest shares that are congruent modulo s' to the "real" shares derived from a secret in $\mathcal{D}_q$. These simulated shares are only used to exponentiate class-group elements, and our proof ensures that all such elements lie in $\langle \mathfrak{g}_\mathfrak{q} \rangle$. Consequently, the simulated shares produce no functional difference in the signing execution.
- Change 2: let ϕ_{K_k} be drawn from a (nearly) uniform distribution over $\mathfrak{G}$, instead of over $\langle \mathfrak{g}_\mathfrak{q} \rangle$. By the HSM assumption, the distribution of ϕ_{K_k} is indistinguishable before and after this change, and ϕ_{K_k} now lies in $\mathfrak{G} \setminus \langle \mathfrak{g}_\mathfrak{q} \rangle$ with probability $1 - \frac{1}{q}$. By the smoothness lemma, $\phi_{K_k}^{v_k} \mathfrak{f}^{x_k}$ is indistinguishable

from $\{\phi_{K_k}^{v_k}\mathfrak{f}^{\tilde{x}} \mid \tilde{x} \xleftarrow{\$} \mathbb{Z}_q\}$ given $C_{i,0} = \mathfrak{g}_\mathfrak{q}{}^{v_k}$ and ϕ_{K_k}, and thus no information about x_k can be inferred from $\phi_{K_k}^{\delta_k} \cdot \phi_{K_k}^{v_k}\mathfrak{f}^{x_k}$. This observation motivates our final modification: replacing $\phi_{K_k}^{v_k}\mathfrak{f}^{x_k}$ with a simulatable value $\phi_{K_k}^{v_k}\mathfrak{f}^{\tilde{x}}$ (independent of x). But it also implicitly alters v_k to a value congruent modulo s' but not necessarily modulo q. While this distinction affects the computation of $\phi_{K^\gamma}^{sk_i'} \in \langle \mathfrak{g}_\mathfrak{q} \rangle \times \langle \mathfrak{f} \rangle$, it is mitigated by our random $\mathfrak{f}^{\alpha_i}$.

With this proof sketch completed, we now present the simulator construction in the ideal world. The analysis of indistinguishability between real and ideal worlds appears in Appendix C.

Proof. WLOG, let $\mathcal{A}$ be an adversary attacking our threshold BBS+ protocol, who corrupts up to $t-1$ parties at the beginning of the execution. Denote by $\mathbf{P}^*$ the corrupted index set and $\overline{\mathbf{P}^*} := [n] \setminus \mathbf{P}^*$ the honest index set, representing corrupted and honest parties respectively.

We construct a simulator $\mathcal{S}$ that invokes $\mathcal{A}$ while emulating honest parties and hybrid functionalities, interacting directly with corrupted parties. Crucially, $\mathcal{S}$ employs the simulation algorithm $\Pi_\mathcal{R}.\mathsf{Sim}(\cdot)$ to generate proofs for honest parties instead of executing $\Pi_\mathcal{R}.\mathsf{Prv}(\cdot,\cdot)$, enabling proof generation without valid witnesses. We omit explicit description of proof generation/broadcasting/verification processes, assuming protocol abortion upon any verification/check failure.

Simulating key generation: $\mathcal{S}$ receives $(\mathbf{H}, X)$ from $\mathcal{F}_{\mathsf{BBS+}}$, fixes an arbitrary honest index $k \in \overline{\mathbf{P}^*}$, and proceeds as follows:

(1) Generate $pp_{cl} \leftarrow \mathsf{CL.Gen}(1^\lambda, q)$ and acts as $\mathcal{F}_{\mathsf{cl}}$ to distribute it.

(2) For the honest index k, sample the secret $v_k \leftarrow \mathcal{D}$, but generate the corrupted parties' shares $\{y_{k,j}\}_{j\in\mathbf{P}^*}$ from another random value $\tilde{v}_k \leftarrow \mathcal{D}_q$. Apply Feldman's simulation to compute $\{C_{k,p}\}_{p=1}^{t-1}$ with $C_{k,0} = \mathfrak{g}_\mathfrak{q}{}^{v_k}$ and $\{y_{k,j}\}_{j\in\mathbf{P}^*}$. Compute the honest parties' shares via Lagrange interpolation:

$$y_{k,i} := \frac{1}{\Delta} \cdot \big(\Delta L_0^{0\cup\mathbf{P}^*}(i) \cdot \Delta v_k + \sum_{j\in\mathbf{P}^*} \Delta L_j^{0\cup\mathbf{P}^*}(i) \cdot y_{k,j}\big).$$

For honest indices $i \neq k$, generate secrets v_i and perform the corresponding PVSS execution. Finally, extract the secrets v_j from corrupted parties to derive $sk := \sum_{j\in[n]} v_j$.

(3) Simulate the honest parties and $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}}$ to correctly compute (sk_i, D_i) for the honest parties while extracting the corrupted parties' shares $\{sk_j\}_{j\in\mathbf{P}^*}$.

(4) When releasing $\{(H_i, X_i)\}_{i\in[n]}$ to construct $(\mathbf{H}, X)$, sample $(\mathbf{H}_i, X_i)$ correctly for honest indices $i \neq k$, and set $(\mathbf{H}_k := \mathbf{H} - \sum_{j\in[n]\wedge j\neq k} \mathbf{H}_j,\ X_k := X - \sum_{j\in[n]\wedge j\neq k} X_j)$ by manipulating $\mathcal{F}_{\mathsf{COM}}$, ensuring that the resulting verification key matches the expected value. When releasing $\{(\phi_{K_i}, \overline{x_i^{K_i}})\}_{i\in[n]}$ to construct $(\phi_K, \overline{x^K})$, correctly compute $(\phi_{K_i}, \overline{x_i^{K_i}})$ for all honest indices $i \neq k$, and set $(\phi_{K_k} := \mathfrak{g}^{r_k},\ \overline{x_k^{K_k}} := \phi_{K_k}^{sk}\mathfrak{f}^{\tilde{x}_k})$ with $r_k \leftarrow \mathcal{D}'$ and $\tilde{x}_k \xleftarrow{\$} \mathbb{Z}_q$.

Simulating signing: Let $\mathcal{T}^*$ denote the set of corrupted signer indices and $\overline{\mathcal{T}^*} := \mathcal{T} \setminus \mathcal{T}^*$ the set of honest signer indices. $\mathcal{S}$ receives (A, e, s) from $\mathcal{F}_{\mathsf{BBS+}}$, fixes an arbitrary new honest index $k \in \overline{\mathcal{T}^*}$, and proceeds as follows:

(5) Broadcast $\phi_{K^{\gamma_i}} := \phi_K^{\gamma_i} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ for $\gamma_i \leftarrow \mathcal{D}$ for every honest index i, while extracting γ_j in the form (γ'_j, ρ_j) using $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}}$ for all corrupted indices j. ϕ_{K^γ} is then computed as $\prod_{j \in \mathcal{T}} \phi_{K^{\gamma_j}}$, consistent with the real protocol.

(6) When releasing $\{(e_i, s_i)\}_{i \in \mathcal{T}}$ to construct (e, s), obtain (e, s) from $\mathcal{F}_{\mathsf{BBS+}}$ and sample (e_i, s_i) correctly for all honest indices $i \neq k$. Then manipulate $\mathcal{F}_{\mathsf{COM}}$ to set $e_k := e - \sum_{j \in [n] \wedge j \neq k} e_j$ and $s_k := s - \sum_{j \in [n] \wedge j \neq k} s_j$, ensuring that the resulting (e, s) match the expected values. Finally, use s to compute $B = G_1 + s \cdot H_1 + \sum_{i \in [\ell]} m_i \cdot H_i$.

(7) For every corrupted index $j \in \mathcal{T}^*$, play as $\mathcal{F}_{\mathsf{zero}}$ to distribute α_j to P_j, then deduce[6] the values sk'_j, the partial decryption pd_j, and the scaled elliptic-curve point R_j as $sk'_j := \Delta L_j^{\mathcal{T}}(0) \cdot sk_j$, $\overline{\gamma_j (x+e)^{K^{\gamma_j}}} := \left(\overline{x^K} \mathfrak{f}^e\right)^{\gamma'_j 2^{-\rho_j}}$, $pd_j := \frac{\overline{\gamma_j (x+e)^{K^{\gamma_j}}}^{\Delta^2}}{\phi_{K^\gamma}^{sk'_j} \cdot \mathfrak{f}^{\alpha_j}}$, and $R_j := (\gamma'_j \cdot 2^{-\rho_j}) \cdot B$.

(8) Given A from $\mathcal{F}_{\mathsf{BBS+}}$, correctly generate (pd_i, R_i) for all honest indices $i \neq k$ according to the protocol, while computing $pd_k := \frac{\mathfrak{f}^{\Delta^2 u}}{\prod_{j \in \mathcal{T} \setminus \{k\}} pd_j}$ and $R_k := u \cdot A - \sum_{j \in \mathcal{T} \setminus \{k\}} R_j$ for the honest index k, with $u \xleftarrow{\$} \mathbb{Z}_q$. Broadcast $\{(pd_i, R_i)\}_{i \in \overline{\mathcal{T}^*}}$ for honest parties and receive $\{(pd_j^*, R_j^*)\}_{j \in \overline{\mathcal{T}^*}}$ from corrupted parties. Derive the signature using these values and abort if the final-round verification fails.

Having established the simulation, the remaining proof is provided in Appendix C, where we present a sequence of hybrid games to show that the gap between the real-world and ideal-world distributions is negligible.

4 Extensions

In this section, we explore extensions of our core threshold BBS+ protocol. Specifically, we provide high-level overviews of how to adapt our techniques to construct threshold BBS signatures and oblivious DY VRFs. The formal protocol details, including concrete specifications and corresponding algorithms, are deferred to Appendix D.

4.1 Threshold BBS signatures

Tessaro and Zhu [29] proved the security of a variant of BBS+, and the resulting scheme essentially matches the original BBS construction implicitly defined

[6] "Deduce" here means extracting the expected value (in the case that the corrupted party behaves honestly) using known information.

in [2]. Throughout, we refer to the "BBS signature scheme" as the one defined in [29]. Compared with BBS+ signatures, BBS signatures are shorter. Although BBS signatures only work over type-3 pairing groups[7], type-3 pairing groups are most commonly used in practice due to their highly-efficient implementations. Furthermore, Tessaro and Zhu proved the security of a variant of BBS signatures in the algebraic group model (AGM) [21], which yields signatures that are even shorter than BBS signatures. In the following, we show how to thresholdize BBS signatures and the AGM-secure variant.

BBS signatures. Compared to BBS+, the BBS signature scheme eliminates the nonce s and reduces the number of $\mathbb{G}_1$-elements in $\mathbf{H}$ from $\ell+1$ to ℓ. For a message $\mathbf{m} \in \mathbb{Z}_q^\ell$, a BBS signature consists of (A, e), where

$$A := \frac{G_1 + \sum_{i\in[\ell]} m_i \cdot H_i}{x+e}.$$

Our threshold BBS+ protocol can be easily adapted to threshold BBS by removing s from the commit-and-release procedure and redefining B as $G_1 + \sum_{i\in[\ell]} m_i \cdot H_i$. The resulting signing protocol requires 3 rounds, matching our threshold BBS+ protocol.

AGM-secure BBS Signatures. Compared to the BBS signature scheme, the AGM-secure BBS variant maintains the same signature format but permits shorter nonces e. In this variant, e is deterministically generated via algorithm $\mathsf{eS}(\mathsf{st}_e, \mathsf{cnt})$, where st_e denotes a state generated via a probabilistic algorithm during key generation and cnt represents a signing counter. The algorithm eS requires only collision resistance for $\mathsf{cnt} \in [p]$, where p denotes the maximum number of signing executions. A typical instantiation sets $\mathsf{eS}(\mathsf{st}_e, \mathsf{cnt}) \to e := \mathsf{cnt}$, yielding e with bit-length $|p| < |q|$.

We adapt the threshold BBS to the AGM-secure variant with the following changes: during key generation, parties run a 2-round commit-and-release procedure to establish the randomness[8] for generating st_e, and during signing, e is computed locally via $\mathsf{eS}(\cdot)$ rather than contributed by multiple signers. The resulting signing protocol completes in 2 rounds, one fewer than both our threshold BBS+ and BBS.

Notably, since synchronizing signing counters across all parties may be challenging, we can implement eS in alternative ways to ensure distinct e per signing execution. For example, each party maintains per-set counters $\mathsf{cnt}_\mathcal{T}$ for each signer index set $\mathcal{T}$ he participates in, then implements eS via a collision-resistant hash function $H(\cdot)$ such that $e \leftarrow H(\mathcal{T}||\mathsf{cnt}_\mathcal{T})$. Here, the hash function $H(\cdot)$ outputs 2λ-bit strings, which is still shorter than $|q|$ in many type-3 pairing curves (e.g., BN462 and BLS48_581).

[7] Type-3 pairing groups require that there is no efficient isomorphism from $\mathbb{G}_2$ to $\mathbb{G}_1$.

[8] We assume security remains intact even if the randomness r used to generate st_e is revealed, as this holds in multiple scenarios; for example, when e is set to cnt (where $r = \emptyset$) or when st_e is directly assigned as r, serving as a PRF seed.

4.2 Oblivious Threshold DY VRF Evaluation

A verifiable random function (VRF) [26] is a pseudorandom function that, on input e, returns a value together with a proof of correct evaluation. Dodis and Yampolskiy introduced the DY VRF [17] and instantiated it over a bilinear group $\mathcal{G}$. Let $(x, X := x \cdot G_2)$ be the secret/public key pair; the VRF output is defined as $F_x(e) := \frac{\mathsf{e}(G_1,G_2)}{x+e}$. The proof is $\pi_e := \frac{G_1}{x+e}$, which enables verification via $\mathsf{e}(\pi_e, X + e \cdot G_2) = \mathsf{e}(G_1, G_2)$ and recovery of the output via $\mathsf{e}(\pi_e, G_2) = F_x(e)$. Thus, given an input e, threshold DY VRF evaluation reduces to jointly computing π_e under a distributed x. Leveraging the structural similarity between DY VRF and AGM-secure BBS, we can obtain a threshold DY VRF. Our goal, however, is to realize *oblivious* threshold DY VRF evaluation, following DKL+23.

In the oblivious VRF evaluation, a user provides the input and interacts with a subset of key-holders (the evaluators) to obtain the output; the requirement is that both the input and the output remain hidden from all key-holders $P_1, \ldots, P_n$, even under full corruption. We highlight the high-level differences between our BBS/BBS+ signing and our oblivious VRF evaluation:

- BBS/BBS+: Signers operate over the ciphertext of x to derive a ciphertext of $\gamma_i(x + e)$, and then broadcast partial decryptions to recover $\gamma(x + e)$.
- Oblivious VRF evaluation: Evaluators operate over the ciphertext of $\eta(x+e)$, provided by the user with a secret value η that is chosen uniformly random, to derive a ciphertext of $\gamma_i\eta(x + e)$; they then broadcast partial decryptions to recover $\gamma\eta(x + e)$.

The specific value η serves as a blinding factor, ensuring that e remains hidden even if the plaintexts implied by these ciphertexts are revealed (which is the case of the full corruption). We now provide full details:

- The user samples $\eta \xleftarrow{\$} \mathcal{D}$ and sends $(\phi_{K^\eta} := \phi_K^\eta, \overline{\eta(x + e)^{K^\eta}} := \left(\overline{x^K} \cdot \overline{e^1}\right)^\eta)$ to the evaluators, along with a ZKPoK of validity. The ZKPoK prevents invalid evaluation requests from malicious users.
- Evaluators verify the ZKPoK. Upon successful verification, they perform homomorphic scaling to derive $(\phi_{K^{\gamma_i\eta}}, \overline{\gamma_i\eta(x + e)^{K^{\gamma_i\eta}}})$ and broadcast $\phi_{K^{\gamma_i\eta}}$ together with a ZKP attesting to its correctness.
- Evaluators send $R_i := \gamma_i \cdot G_1$ and partial decryptions for recovering $u = \gamma\eta(x + e)$ to the user, enabling the user to derive $\pi_e = \frac{\sum_{j\in\mathcal{T}} R_j}{(u/\eta)}$.

The ZKPoK for $(\phi_{K^\eta}, \overline{\eta(x + e)^{K^\eta}})$ checks the following relation:

$$\mathcal{R}_{\mathsf{scal\text{-}add}} := \left\{ \begin{array}{l} ((c_1, c_2, c_3, c_4), (\alpha, \beta)) : \\ c_3 = c_1^\alpha \wedge c_4 = c_2^\alpha \mathfrak{f}^\beta \end{array} \right\},$$

where $(c_1, c_2, c_3, c_4, \alpha, \beta)$ are assigned as $(\phi_K, \overline{x^K}, \phi_{K^\eta}, \overline{\eta(x + e)^{K^\eta}}, \eta, \eta e)$. The corresponding ZKPoK protocol is presented in Appendix B.2. Applying the similar extraction technique used for γ_i in our threshold BBS+, the above η is extracted in the form (η', ρ) such that $\eta = \eta' \cdot 2^{-\rho}$. A side effect of the above

protocol is that we no longer realize $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}}$ for extracting γ_i, so $\phi_{K^{\gamma_i\eta}} = (\phi_{K^\eta})^{\gamma_i}$ is now validated via a ZKP. This is because ϕ_{K^η} is not uniformly distributed in $\langle \mathfrak{g}_\mathfrak{q} \rangle$ when η is chosen by malicious users. Nevertheless, the protocol remains applicable for extracting $\gamma_i\eta$ using the previously obtained η, which suffices for security since $\gamma_i\eta$ enables simulation and deduction of $\overline{\gamma_i\eta(x+e)^{K^{\gamma_i\eta}}} = \left(\overline{x^K} \cdot \overline{e^1}\right)^{\gamma_i\eta}$. Details of the $\gamma_i\eta$ extraction appear in Appendix B.3.

Regarding the communication–computation trade-off, our oblivious DY VRF evaluation mirrors threshold BBS+ when compared with DKL+23, exhibiting lower communication overhead at the cost of higher computational overhead. Moreover, it completes in only 2 rounds (one round to broadcast $\phi_{K^{\gamma_i\eta}}$ and one round to recover u), which is one fewer than DKL+23's 3-round protocol (two rounds for MtA and one round to recover u).

5 Discussions of Our Modified Homomorphic Scaling

In this section, we extend our technique to the multiplication protocols in recent two-round threshold ECDSA schemes DNP25 [15] and LLZ+25 [25]. After stating the problem, we describe their solutions and present our modifications.

Consider parties P_i indexed by $\mathcal{T}$, where each holds $(a_i, b_i) \in \mathbb{Z}_q$. The goal is to securely compute $a \cdot b$ for $a := \sum_{j\in\mathcal{T}} a_j$ and $b := \sum_{j\in\mathcal{T}} b_j$. The protocols operate as follows:

- Initialize $g_0, g_1 \in \langle \mathfrak{g}_\mathfrak{q} \rangle$. For DNP25, $g_0 = \mathfrak{g}_\mathfrak{q}$ and $g_1 = pk$; for LLZ+25, g_0 and g_1 are random elements in $\langle \mathfrak{g}_\mathfrak{q} \rangle$.
- Each P_i samples $\alpha_i, \beta_i \leftarrow \mathcal{D}_q$, broadcasts a Pedersen commitment $A_i := g_0^{\alpha_i} g_1^{a_i}$ and a CL-ciphertext-like tuple $(B_{i,0} := g_0^{\beta_i},\ B_{i,1} := g_1^{\beta_i}\mathfrak{f}^{b_i})$. Note that ZKPoKs for validating $(A_i, B_{i,0}, C_{i,0})$ are also broadcast when DNP25 and LLZ+25 employ their multiplication protocols in their threshold ECDSA schemes.
- *Derivation in DNP25*: Define $\alpha := \sum_{j\in\mathcal{T}} \alpha_j$, $\beta := \sum_{j\in\mathcal{T}} \beta_j$. Parties compute $A := \prod_{j\in\mathcal{T}} A_j = g_0^\alpha g_1^a$, $B_0 := \prod_{j\in\mathcal{T}} B_{j,0} = g_0^\beta$, $B_1 := \prod_{j\in\mathcal{T}} B_{j,1} = g_1^\beta \mathfrak{f}^b$. Each P_i broadcasts $C_i := B_1^{a_i} B_0^{\alpha_i} / A^{\beta_i}$. Then ab is derived as:

$$ab = \mathsf{CL.Solve}\left(\prod_{j\in\mathcal{T}} C_j\right) = \mathsf{CL.Solve}\left(\frac{B_1^a B_0^\alpha}{A^\beta}\right) = \mathsf{CL.Solve}\left(\frac{g_1^{a\beta}\mathfrak{f}^{ab} g_0^{\alpha\beta}}{g_0^{\alpha\beta} g_1^{a\beta}}\right).$$

- *Derivation in LLZ+25*: For $i \neq j$, P_i and P_j locally share $a_i b_j$ via function $\psi(\cdot)$ where $\psi(\eta) = \psi(\eta\mathfrak{f}^e) \in \{\eta\mathfrak{f}^\mu \mid \mu \in \mathbb{Z}_q\}$ for any $\eta \in \mathfrak{G}$ and $e \in \mathbb{Z}_q$. P_i computes $C_{i,j,0} := B_{j,0}^{\alpha_i} B_{j,1}^{a_i}$ and derives $c_{i,j,0} := \mathsf{CL.Solve}(C_{i,j,0}/\psi(C_{i,j,0}))$. Concurrently, P_j computes $C_{i,j,1} := A_i^{\beta_j}$ and derives $c_{i,j,1} := \mathsf{CL.Solve}(\psi(C_{i,j,1})/C_{i,j,1})$. Verification shows:

$$c_{i,j,0} + c_{i,j,1} = \mathsf{CL.Solve}\left(\frac{C_{i,j,0}}{\psi(C_{i,j,0})} \cdot \frac{\psi(C_{i,j,1})}{C_{i,j,1}}\right) = \mathsf{CL.Solve}\left(\mathfrak{f}^{a_i b_j}\right) = a_i b_j.$$

  Each P_i broadcasts $c_i := a_i b_i + \sum_{j\in\mathcal{T}\setminus\{i\}} (c_{i,j,0} + c_{j,i,1})$, yielding $ab = \sum_{j\in\mathcal{T}} c_j$.

Applying our technique. Recall from Section 1.3 that the first element of a scaled CL ciphertext is $\phi_{K'} := \phi_K^{w_1} \mathfrak{g_q}^{w_2}$ (using (w_1, w_2) instead of (a, r) for clarity). Interpreting (g_0, g_1) as $(\mathfrak{g_q}, \phi_K)$, $\phi_{K'}$ matches A_i's structure. We thus modify the homomorphic scaling for A_i as:

$$A_i := g_1^{a_i + q\alpha_i}.$$

The commitment properties are preserved:

- *Hiding*: Follows from $\phi_{K^*} := \phi_K^{w_1 + w_2 q}$ concealing w_1, relying on $\gcd(s', q) = 1$.
- *Binding*: Guaranteed by the ZKPoK of A_i extracting $a_i + q\alpha_i$. Finding $w \neq a_i + q\alpha_i$ that satisfies $g_1^w = A_i$ implies $g_1^{w-(a_i+q\alpha_i)} = 1$, violating the Unknown Order Assumption [4, Definition 1].

Modified derivations. Denoting $a_i + q\alpha_i$ by a_i', we derive ab as:

- *Modified DNP25*: Parties compute $A := \prod_{j\in\mathcal{T}} A_j = g_1^{\sum_{j\in\mathcal{T}} a_i'}$ and $B_1 := \prod_{j\in\mathcal{T}} B_{j,1} = g_1^{\beta} \mathfrak{f}^b$. Each P_i broadcasts $C_i := B_1^{a_i'}/A^{\beta_i}$, then

$$ab = \mathsf{CL.Solve}(\prod_{j\in\mathcal{T}} C_j).$$

- *Modified LLZ+25*: For $i \neq j$, P_i and P_j compute $C_{i,j,0} := B_{j,1}^{a_i'} = g_1^{a_i'\beta_j} f^{a_i' b_j}$ and $C_{i,j,1} := A_i^{\beta_j} = g_1^{a_i'\beta_j}$, deriving $c_{i,j,0}$, $c_{i,j,1}$ similarly. Correctness holds as

$$\frac{C_{i,j,0}}{\psi(C_{i,j,0})} \cdot \frac{\psi(C_{i,j,1})}{C_{i,j,1}} = C_{i,j,0}/C_{i,j,1} = \mathfrak{f}^{a_i' b_j} = \mathfrak{f}^{a_i b_j}.$$

The value ab is computed via the same procedure as the original approach.

Notably, $B_{i,0}$ becomes unnecessary and not broadcast in the modified protocols. Each party thus broadcasts one fewer class-group element and simplifies the ZKPoK relation, reducing communication and computation while maintaining security, as the standalone $B_{i,1}$ leaks strictly less information than $(B_{i,0}, B_{i,1})$.

Compare with our method for the threshold BBS+ scenario. For simplicity, assume $e = 0$ so multiplication simplifies to γx. As demonstrated in modified DNP25 and LLZ+25, generating either A_i or $B_{i,1}$ for γ_i requires a ZKPoK at least as expensive as our ZKPoK for proving $\phi_{K^{\gamma_i}} = \phi_K^{\gamma_i}$. Additionally:

- *Modified DNP25*: Following their threshold ECDSA approach, information of x are previously stored (during key generation) in $\{B_{i,1}' = g_1^{\beta_i'} f^{x_i}\}_{i\in[n]}$ where x_i is a Shamir secret share of x. This enables reconstruction via $x = \sum_{j\in\mathcal{T}} L_j^{\mathcal{T}}(0) x_j$, requiring computation of $B_1 = \prod_{j\in\mathcal{T}} B_{j,1}^{L_j^{\mathcal{T}}(0)}$ (where previous β_i and b_i correspond to $L_i^{\mathcal{T}}(0)\beta_i'$ and $L_i^{\mathcal{T}}(0)x_i$ respectively). This incurs $\sim t$ additional exponentiations.
- *Modified LLZ+25*: Computing each $C_{i,j,0}$ and $C_{i,j,1}$ would require $\sim 2t$ additional exponentiations.

Thus, our protocol maintains the best signing performance (number of exponentiations: our $\sim 2t$ v.s. DNP25's $\sim 3t$ v.s. LLZ+25's $\sim 4t$).

	Out. Comm. Cost $\mathsf{O}_{\mathsf{cost}}$	In. Comm. Cost $\mathsf{I}_{\mathsf{cost}}$
DKL+23:	$4\lambda + 2\kappa + (t-1)(\frac{7}{2}\lambda^2 + 6\kappa^2 + \lambda\lambda_d + \lambda\kappa + 6\lambda_d\kappa + \frac{7}{2}\lambda + \kappa)$	$\approx \mathsf{O}_{\mathsf{cost}}$
WMC24:	$13\lambda + 13\lambda_d + 12\kappa + 9\lvert\hat{\mathfrak{G}}\rvert + 6\lvert\overline{s}\rvert + 3\lvert\mathbb{G}_1\rvert$	$(t-1)\mathsf{O}_{\mathsf{cost}}$
Our work:	$5\lambda + 2\lambda_d + 3\kappa + 2\lvert\hat{\mathfrak{G}}\rvert + \lvert\overline{s}\rvert + \lvert\mathbb{G}_1\rvert$	$(t-1)\mathsf{O}_{\mathsf{cost}}$

Table 2: Theoretical communication cost per signer in the (t, n) threshold setting. Parameters κ, $|\hat{\mathfrak{G}}|$, and $|\Gamma|$ denote the respective element sizes in the groups $\mathbb{Z}_q$, $\hat{\mathfrak{G}}$, and $\mathbb{G}_1$.

6 Performance Evaluation and Experimental Analysis

6.1 Communication Overhead—Bandwidth Requirement

We present a theoretical comparison on communication cost between DKL+23, WMC24 and our work in Table 2. We note that DKL+23 employs a distinct communication model from WMC24 and ours: in DKL+23, signers transmit the final-round message for deriving the signature directly to a client without broadcasting to signers, whereas all messages are broadcast to all signers in both our protocol and WMC24. Since our table exclusively records inter-party communication costs, we have excluded DKL+23's final round from the corresponding entry. This omission renders our comparison more favorable to DKL+23. Additionally, when calculating outgoing communication, we attribute only a single transmission cost for messages sent via broadcast channels. Consequently, our computed outgoing communication for DKL+23 is marginally lower than their reported value, though our incoming communication cost calculation remains consistent with their data.

For a more direct comparison, Table 3 summarizes the concrete communication costs across all three schemes for $t \in \{2, 5, 10, 15, 20\}$ signers. Note that the parameter n, representing the total number of parties, does not affect signing performance. We now detail the associated parameter values. The bilinear group is chosen as BLS12-381 [27], with parameters $\lambda = 128$, $\lambda_d = 80$, $\kappa = 256$ and $|\mathbb{G}_1| = 384$ bits. Each $\hat{G}$-element is in compressed form [10] with size 1752 bits, and the respective $\overline{s}$ requires 936 bits.[9]

6.2 Implementation and Performance Evaluation

We implemented our threshold BBS+ protocol in Rust using parameters consistent with our bandwidth overhead analysis. For CL framework operations, we adapted the practical implementation from [3] into Rust.

To enable direct comparison with DKL+23's reported data (whose implementation was not released), we replicated their experimental environment with the following configuration:

[9] $\overline{s} = \left\lceil \frac{1}{\pi} \log |\Delta_K| \cdot \sqrt{|\Delta_K|} \right\rceil$, where Δ_K has a bit length of 1827 for 128-bit security.

	Outgoing communication cost(KiB)					Incoming communication cost(KiB)				
	$t=2$	$t=5$	$t=10$	$t=15$	$t=20$	$t=2$	$t=5$	$t=10$	$t=15$	$t=20$
DKL+23	75.46	301.46	678.14	1054.83	1431.51	75.46	301.84	679.14	1056.45	1433.76
WMC24	3.45	3.45	3.45	3.45	3.45	3.45	13.82	31.10	48.38	65.66
Our work	0.78	0.78	0.78	0.78	0.78	0.78	3.12	7.02	10.92	14.83

Table 3: Concrete incoming-outgoing communication cost per signer in the (t, n) threshold setting with the BLS12_381 curve.

	Multi-threaded Performance(ms)					Single-threaded Performance(ms)				
	$t=2$	$t=5$	$t=10$	$t=15$	$t=20$	$t=2$	$t=5$	$t=10$	$t=15$	$t=20$
DKL+23	∼3.0	∼4.0	∼6.3	∼9.4	∼12.2	-	-	-	-	-
WMC24	149.4	169.9	200.4	231.0	263.3	677.9	1590.9	3112.7	4634.9	6157.2
Our work	45	49	47.9	47.6	48.5	64.0	114.6	199.5	284.8	370.6

Table 4: Concrete computation cost per signer in the (t, n) threshold setting with the BLS12_381 curve.

- All tests executed on Google Cloud `c2d-standard-4` instances (4 vCPUs: AMD EPYC 7B13, 16GB RAM) scaling from 2 to 20 signers, with 10 Gbps bandwidth cap.
- Servers located in the `us-east1-c` zone.
- Performance metrics recording maximum average time from request receipt to signature generation.

While DKL+23 provide only multi-threaded results, we conducted both multi-threaded and single-threaded tests for our protocol and WMC24 to enable comprehensive analysis. The concrete comparison is presented in Table 4. In addition, we include the experiments on the key generation protocol in Appendix A.1 and a comparison with TX25 [28] in Appendix A.2.

Comparison with DKL+23. Benchmarks show our protocol runs 4-16× slower than DKL+23, with the slowdown factor asymptotically tightening to 4×. This primarily stems from the highly efficient OT-based implementation of MtA in DKL+23. However, as Table 3 demonstrates, our protocol achieves $\sim 100\times$ better communication efficiency. Consequently, we contend our solution provides a better computation-communication trade-off.

Remark 6. DKL+23 also presents experimental results under WAN setting using Google Cloud servers distributed across the US and Europe zones. To ensure a fair comparison, we replicate their experimental setup. Given that their work does not specify the exact server distribution, we adopt a conservative approach

by assuming maximum theoretical latency conditions—setting up servers as uniformly distributed across their mentioned regions. Under these unfavorable network assumptions, our protocol demonstrates superior performance: 114ms at $t = 4$ (v.s. DKL+23's ∼140ms), where one server in each US region; 120ms at $t = 8$ (v.s. ∼180ms), where two servers in each US region; 229ms at $t = 15$ (v.s. DKL+23's ∼340ms), where three servers in each US region and one server in each Europe region. This performance advantage arises primarily from our protocol's reduced communication overhead, which proves more efficient in WAN.

Comparison with WMC24. Benchmarks show our protocol achieves a 3.3–5.4× speedup in multi-threaded execution and 10.6–16.6× in single-threaded mode, approaching 16.6× as t increases. The single-threaded results match theoretical expectations: $\sim 2t$ exponentiations versus WMC24's $\sim 29t$.

Acknowledgements

This work is partially supported by the Research Grant Council of Hong Kong (Project No.: 15230125, C1029-22GF, R1012-21), Innovation and Technology Fund of the Innovation and Technology Commission of Hong Kong (Project No.: ITS/176/23FP), The Hong Kong Polytechnic University (Project No.: P0048350), and in part by the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grants (Proposal ID: 23-SIS-SMU-035, 24-SIS-SMU-052).

References

1. Au, M.H., Susilo, W., Mu, Y.: Constant-size dynamic k-taa. In: Security and Cryptography for Networks: 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006. Proceedings 5. pp. 111–125. Springer (2006)
2. Boneh, D., Boyen, X., Shacham, H.: Short group signatures. In: Annual international cryptology conference. pp. 41–55. Springer (2004)
3. Bouvier, C., Castagnos, G., Imbert, L., Laguillaumie, F.: I want to ride my bicycl: Bicycl implements cryptography in class groups. Journal of Cryptology **36**(3), 17 (2023)
4. Braun, L., Damgård, I., Orlandi, C.: Secure multiparty computation from threshold encryption based on class groups. In: Annual International Cryptology Conference. pp. 613–645. Springer (2023)
5. Brickell, E., Li, J.: Enhanced privacy id from bilinear pairing. Cryptology ePrint Archive (2009)
6. Camenisch, J., Drijvers, M., Lehmann, A.: Anonymous attestation using the strong diffie hellman assumption revisited. In: Trust and Trustworthy Computing: 9th International Conference, TRUST 2016, Vienna, Austria, August 29-30, 2016, Proceedings 9. pp. 1–20. Springer (2016)
7. Camenisch, J., Lysyanskaya, A.: A signature scheme with efficient protocols. In: Security in Communication Networks: Third International Conference, SCN 2002 Amalfi, Italy, September 11–13, 2002 Revised Papers 3. pp. 268–289. Springer (2003)
8. Camenisch, J., Lysyanskaya, A.: Signature schemes and anonymous credentials from bilinear maps. In: Annual international cryptology conference. pp. 56–72. Springer (2004)
9. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145. IEEE (2001)
10. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Two-party ecdsa from hash proof systems and efficient instantiations. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39. pp. 191–221. Springer (2019)
11. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Bandwidth-efficient threshold ec-dsa. In: IACR International Conference on Public-Key Cryptography. pp. 266–296. Springer (2020)
12. Castagnos, G., Laguillaumie, F.: Linearly homomorphic encryption from ddh. In: Cryptographers' Track at the RSA Conference. pp. 487–505. Springer (2015)
13. Castagnos, G., Laguillaumie, F., Tucker, I.: Practical fully secure unrestricted inner product functional encryption modulo p. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 733–764. Springer (2018)
14. Cramer, R., Damgård, I., Nielsen, J.B.: Multiparty computation from threshold homomorphic encryption. In: International conference on the theory and applications of cryptographic techniques. pp. 280–300. Springer (2001)
15. Dahari-Garbian, H., Nof, A., Parker, L.: Trout: Two-round threshold ecdsa from class groups. Cryptology ePrint Archive (2025)
16. Desmedt, Y.: Threshold cryptosystems. In: International Workshop on the Theory and Application of Cryptographic Techniques. pp. 1–14. Springer (1992)

17. Dodis, Y., Yampolskiy, A.: A verifiable random function with short proofs and keys. In: International Workshop on Public Key Cryptography. pp. 416–431. Springer (2005)
18. Doerner, J., Kondi, Y., Lee, E., Shelat, A., Tyner, L.: Threshold bbs+ signatures for distributed anonymous credential issuance. In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 773–789. IEEE (2023)
19. Faust, S., Hazay, C., Kretzler, D., Rometsch, L., Schlosser, B.: Non-interactive threshold bbs+ from pseudorandom correlations. In: Cryptographers' Track at the RSA Conference. pp. 198–222. Springer (2025)
20. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
21. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II 38. pp. 33–62. Springer (2018)
22. Gennaro, R., Goldfeder, S., Ithurburn, B.: Fully distributed group signatures. See orbs. com/white-papers/fully-distributed-group-signatures/website (2019)
23. Hesse, J., Singh, N., Sorniotti, A.: How to bind anonymous credentials to humans. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 3047–3064 (2023)
24. Lindell, Y.: How to simulate it–a tutorial on the simulation proof technique. Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich pp. 277–346 (2017)
25. Lyu, Y., Li, Z., Zhou, H.S., Deng, X.: Threshold ecdsa in two rounds. Cryptology ePrint Archive (2025)
26. Micali, S., Rabin, M., Vadhan, S.: Verifiable random functions. In: 40th annual symposium on foundations of computer science (cat. No. 99CB37039). pp. 120–130. IEEE (1999)
27. Sean, B.: Bls12-381: New zk-snark elliptic curve construction. `https://electriccoin.co/blog/new-snark-curve/` (2017), accessed: 2023-08-20
28. Tang, G., Xue, H.: Robust threshold ecdsa with online-friendly design in three rounds. Cryptology ePrint Archive (2025)
29. Tessaro, S., Zhu, C.: Revisiting bbs signatures. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 691–721. Springer (2023)
30. Tobias Looker, Vasilis Kalos, A.W., Lodder, M.: Signature scheme. internet-draft draft-irtf-cfrg-bbssignatures-08. Internet Engineering Task Force (March 2025)
31. Wong, H.W., Ma, J.P., Chow, S.S.: Secure multiparty computation of threshold signatures made more efficient. In: ISOC Network and Distributed System Security Symposium–NDSS 2024 (2024)
32. Xie, Z., Liu, M., Xue, H., Au, M.H., Deng, R.H., Yiu, S.M.: Direct range proofs for paillier cryptosystem and their applications. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 899–913 (2024)
33. Yuen, T.H., Cui, H., Xie, X.: Compact zero-knowledge proofs for threshold ecdsa with trustless setup. In: IACR International Conference on Public-Key Cryptography. pp. 481–511. Springer (2021)

A Other comparisons

A.1 Comparison on the key generation protocol

We provide key generation performance comparisons (LAN, multi-threaded) between our protocol and DKL+23 in Table 5. The data show that our key generation achieves lower outgoing bandwidth (asymptotically by $> 10\times$) and lower running time than DKL+23, but incurs higher incoming bandwidth. This results from CL-DKG requiring more broadcast data than DKL+23. Since incoming bandwidth scales with n times the broadcast data volume, our key generation's incoming bandwidth grows more substantially with increasing n. Our implementation of WMC24 key generation exhibits similar performance to our protocol and is therefore omitted. However, the original WMC24 key generation protocol is theoretically slower than our implementation, as we employ a simpler construction for generating signing-related information.

		$t=2$	$t=5$	$t=10$	$t=15$	$t=20$
Outgoing Bandwidth(KiB)	DKL+23	16.2	52.2	112.4	172.6	232.7
	Our work	3	5.6	9.9	14.3	18.8
Incoming Bandwidth(KiB)	DKL+23	16.2	64.8	145.8	226.8	307.9
	Our work	3	22.2	89	200.1	356.6
Runtime(s)	DKL+23	∼0.29	∼1.58	∼3.8	∼6	∼8.3
	Our work	0.27	0.54	1.1	1.6	2.2

Table 5: Concrete key generation performance in the threshold (t, n) setting where $t = n$.

A.2 Comparison with TX25

Under TX25 testing environment (a MacBook Pro with macOS Monterey version 12.3, 16GB RAM, and an Apple M1 Pro chip.), we test overall signing runtime for our protocol and WMC24 (Rust implementations), and compared with TX25 and WMC24 (C++ implementations) in TX25 paper. Concrete data is presented in Table 6, showing that our protocol achieves consistently greater speedup over WMC24 than TX25 across all thresholds t: we demonstrate at least $8\times$ speedup (at $t = 2$) with asymptotic factor exceeding $14\times$, while TX25 achieves at most $5.1\times$ (at $t = 2$) and becomes less efficient than WMC24 when $t > 20$.

B Security requirement and realizations of ZKPs

B.1 Full security requirement

A non-interactive ZKP protocol $\Pi_{\mathcal{R}} = (\Pi_{\mathcal{R}}.\mathsf{Prv}, \Pi_{\mathcal{R}}.\mathsf{Vrf})$ for relation $\mathcal{R}$ should satisfy:

	$t=2$	$t=5$	$t=10$	$t=15$	$t=20$
TX25(C++) [18]	0.20s	0.71s	2.25s	4.51s	7.52s
WMC24(C++ [18]	1.02s	2.02s	3.66s	5.31s	6.98s
WMC24(Rust) [31]	0.49s	1.11s	2.14s	3.18s	4.21s
Ours(Rust)	0.06s	0.10s	0.17s	0.24s	0.31s

Table 6: Concrete signing performance for WMC24(C++/Rust), TX(C++) and our protocol(Rust) in the threshold (t, n) setting.

- **Completeness**: For any $(x, w) \in \mathcal{R}$, we have

$$\Pr[\Pi_{\mathcal{R}}.\mathsf{Vrf}(x, \pi) = 1 : \pi \leftarrow \Pi_{\mathcal{R}}.\mathsf{Prv}(x, w)] = 1.$$

- **Standard Soundness**: For any PPT adversary $\mathcal{P}^*$ and $x \notin \mathcal{L}$, the probability

$$\Pr[\Pi_{\mathcal{R}}.\mathsf{Vrf}(x, \pi^*) = 1 : \pi^* \leftarrow \mathcal{P}^*(x)]$$

  is negligible in λ.
- **Honest verifier zero-knowledge (HVZK)**: There exists an algorithm $\Pi_{\mathcal{R}}.\mathsf{Sim}(\cdot)$ that can simulate a proof by programming the random oracle, such that the distributions of $\Pi_{\mathcal{R}}.\mathsf{Sim}(x)$ and $\Pi_{\mathcal{R}}.\mathsf{Prv}(x, w)$ are statistically close in λ_d for any $(x, w) \in \mathcal{R}$.

B.2 ZKP(oK) Protocols

Statement: $pk \in \langle \mathfrak{g}_\mathfrak{q} \rangle, c_1, c_2, M$.
Witness: $m \in \mathbb{Z}_q, r \in \mathcal{D}_q$.
Relation: $c_1 = {\mathfrak{g}_\mathfrak{q}}^r \wedge c_2 = pk^r f^m \wedge M = mG_2$.
The protocol works with prover P and verifier V as follows:

1. P samples $r_1 \xleftarrow{\$} [2^{\lambda+\lambda_d}\mathcal{B}_q]$ and $r_2 \xleftarrow{\$} \mathbb{Z}_q$, computes $c_1' := {\mathfrak{g}_\mathfrak{q}}^{r_1}$, $c_2' := pk^{r_1}\mathfrak{f}^{r_2}$ and $M' := r_2 G_2$, then sends (c_1', c_2', M') to V.
2. Upon receiving (c_1', c_2', M') from P, V samples $k \xleftarrow{\$} [0, 2^\lambda)$ and sends k to P.
3. Upon receiving k from V, P computes and sends $(r_3 := r_1 + kr, r_4 := r_2 + km \bmod q)$ to V.
4. V checks $r_3 \overset{?}{\in} [2^\lambda \mathcal{B}_q(2^{\lambda_d} + 1)]$, ${\mathfrak{g}_\mathfrak{q}}^{r_3} \overset{?}{=} c_1' c_1^k$, $pk^{r_3}\mathfrak{f}^{r_4} \overset{?}{=} c_2' c_2^k$ and $G_2^{r_4} \overset{?}{=} M' + k \cdot M$. If the checks pass, return 1; otherwise, return 0.

Fig. 9: The ZKP protocol for relation $\mathcal{R}_{\mathsf{enc}}$.

In the following, we present only the interactive ZKP(oK) protocols for the specified relations, as their non-interactive versions are generically obtainable via the Fiat-Shamir heuristic:

Statement: $c \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ (near-uniformly distributed), C.
Witness: $w \in [0, \mathcal{B})$.
Relation: $C = c^w$.
The protocol works with prover P and verifier V as follows:

1. P samples $r_1 \xleftarrow{\$} [2^{\lambda+\lambda_d}\mathcal{B}]$, computes $c' := c^{r_1}$, then sends c' to V.
2. Upon receiving t from P, V samples $k \xleftarrow{\$} [0, 2^\lambda)$ and sends k to P.
3. Upon receiving k from V, P computes and sends $r_2 := r_1 + kw$ to V.
4. V checks $r_2 \overset{?}{\in} [2^\lambda \mathcal{B}(2^{\lambda_d} + 1)]$ and $c^{r_2} \overset{?}{=} c'C^k$. If the checks pass, return 1; otherwise, return 0.

Fig. 10: The ZKPoK protocol for relation $\mathcal{R}_{\text{dls}}$.

Statement: $c_1 \in \langle \mathfrak{g}_\mathfrak{q} \rangle, c_2 \in G, c_3, c_4$.
Witness: $\alpha \in [0, \mathcal{B}), \beta \in \mathbb{Z}_q$.
Relation: $c_3 = c_1^\alpha \wedge c_4 = c_2^\alpha \mathfrak{f}^\beta$.
The protocol works with prover P and verifier V as follows:

1. P samples $r_1 \xleftarrow{\$} [2^{\lambda+\lambda_d}\mathcal{B}]$ and $r_2 \xleftarrow{\$} \mathbb{Z}_q$, computes $c_1' := c_1^{r_1}$ and $c_2' := c_2^{r_1}\mathfrak{f}^{r_2}$, then sends (c_1', c_2') to V.
2. Upon receiving (c_1', c_2') from P, V samples $k \xleftarrow{\$} [0, 2^\lambda)$ and sends k to P.
3. Upon receiving k from V, P computes and sends $(r_3 := r_1 + k\alpha, r_4 := r_2 + k\beta \bmod q)$ to V.
4. V checks $r_3 \overset{?}{\in} [2^\lambda \mathcal{B}(2^{\lambda_d} + 1)]$, $c_1^{r_3} \overset{?}{=} c_1' c_3^k$ and $c_2^{r_3}\mathfrak{f}^{r_4} \overset{?}{=} c_2' c_4^k$. If the checks pass, return 1; otherwise, return 0.

Fig. 11: The ZKAoK for relation $\mathcal{R}_{\mathsf{scal\text{-}add}}$.

- $\mathcal{R}_{\text{dlc}}$: Both ZKP and ZKPoK protocols are implemented using [33, Algorithm 4]; see [33] for witness extraction capabilities. Verification requires 5 exponentiations in $G \setminus F$.
- $\mathcal{R}_{\mathsf{enc}}$: We use a Σ protocol for ZKP (Fig. 9). Verification requires 4 exponentiations in $G \setminus F$.
- $\mathcal{R}_{\text{dls}}$: We use a Σ protocol for ZKPoK (Fig. 10). Verification requires 2 exponentiations in $G \setminus F$.
- $\mathcal{R}_{\mathsf{scal\text{-}add}}$: We use a Σ protocol for ZKPoK (Fig. 11). Verification requires 4 exponentiations in $G \setminus F$.

B.3 Extraction of $\gamma_i \eta$

We aim to extract $\gamma_i \eta$ implied in $\phi_{K^\eta}^{\gamma_i} = \phi_{K^{\gamma_i \eta}}$, given (η', ρ) satisfying $\phi_K^{\eta' 2^{-\rho}} = \phi_{K^\eta}$ and valid transcripts (ψ, c_1, r_1), (ψ, c_2, r_2) such that $\psi \phi_{K^{\gamma_i \eta}}^{c_j} = \phi_{K^\eta}^{r_j}$ for $j \in [2]$. Setting $c = 2^\rho(c_1 - c_2)$ and $r = \eta'(r_1 - r_2)$, we obtain $\phi_{K^{\gamma_i \eta}}^c = \phi_K^r$.

Let $d = \gcd(r, c)$ and define $v := \phi_K^{r/d}\phi_{K^{\gamma_i\eta}}^{-c/d}$. We claim $v = 1$; otherwise with overwhelming probability we find $(v \in \hat{\mathfrak{G}} \setminus \mathfrak{F}, d \in \mathbb{Z})$ satisfying $v^d = 1$, contradicting the Unknown Order Assumption [4, Definition 1]. With $v = 1$, we have $\phi_K^{r/d} = \phi_{K^{\gamma_i\eta}}^{c/d}$. Denoting $e = c/d$, find α, β such that $\alpha c + \beta r = d$, yielding $\phi_K^d = (\phi_K^\alpha \phi_{K^{\gamma_i\eta}}^\beta)^c$. Thus $(\phi_K^\alpha \phi_{K^{\gamma_i\eta}}^\beta)^{c/d} = \phi_K$ (otherwise we find $u = (\phi_K^\alpha \phi_{K^{\gamma_i\eta}}^\beta)^{c/d} \cdot \phi_K^{-1} \in \hat{\mathfrak{G}} \setminus \mathfrak{F}$ s.t. $u^d = 1$, contradicting the Unknown Order Assumption). Since ϕ_K is uniformly and unbiasedly distributed in $\langle \mathfrak{g}_\mathfrak{q} \rangle$, if $e \neq 2^t$ for any $t \in \mathbb{Z}$, we break the Strong Root Assumption [11, Definition 10], a contradiction. Therefore $\phi_K^{r/d \cdot 2^{-t}} = \phi_{K^{\gamma_i\eta}}$, extracting $p = r/d$ and t such that $\phi_{K^{\gamma_i\eta}} = \phi_K^{p \cdot 2^{-t}}$.

This technique originates from [11], replacing their Low Order Assumption [11, Definition 9] with the stronger Unknown Order Assumption.

C Security Proof

Proof. WLOG, let $\mathcal{A}$ be an adversary attacking our threshold BBS+ protocol, who corrupts up to $t-1$ parties at the beginning of the execution. Denote by $\mathbf{P}^*$ the corrupted index set and $\overline{\mathbf{P}^*} := [n] \setminus \mathbf{P}^*$ the honest index set, representing corrupted and honest parties respectively.

We construct a simulator $\mathcal{S}$ that invokes $\mathcal{A}$ while emulating honest parties and hybrid functionalities, interacting directly with corrupted parties. Crucially, $\mathcal{S}$ employs the simulation algorithm $\Pi_\mathcal{R}.\mathsf{Sim}(\cdot)$ to generate proofs for honest parties instead of executing $\Pi_\mathcal{R}.\mathsf{Prv}(\cdot,\cdot)$, enabling proof generation without valid witnesses. We omit explicit description of proof generation/broadcasting/verification processes, assuming protocol abortion upon any verification/check failure.

Simulating key generation: $\mathcal{S}$ receives $(\mathbf{H}, X)$ from $\mathcal{F}_{\mathsf{BBS+}}$, fixes an arbitrary honest index $k \in \overline{\mathbf{P}^*}$, and proceeds as follows:

(1) Generate $pp_{cl} \leftarrow \mathsf{CL.Gen}(1^\lambda, q)$ and acts as $\mathcal{F}_{\mathsf{cl}}$ to distribute it.

(2) For the honest index k, sample the secret $v_k \leftarrow \mathcal{D}$, but generate the corrupted parties' shares $\{y_{k,j}\}_{j \in \mathbf{P}^*}$ from another random value $\tilde{v}_k \leftarrow \mathcal{D}_q$. Apply Feldman's simulation to compute $\{C_{k,p}\}_{p=1}^{t-1}$ with $C_{k,0} = {\mathfrak{g}_\mathfrak{q}}^{v_k}$ and $\{y_{k,j}\}_{j \in \mathbf{P}^*}$. Compute the honest parties' shares via Lagrange interpolation:

$$y_{k,i} := \frac{1}{\Delta} \cdot \big(\Delta L_0^{0 \cup \mathbf{P}^*}(i) \cdot \Delta v_k + \sum_{j \in \mathbf{P}^*} \Delta L_j^{0 \cup \mathbf{P}^*}(i) \cdot y_{k,j}\big).$$

For honest indices $i \neq k$, generate secrets v_i and perform the corresponding PVSS execution. Finally, extract the secrets v_j from corrupted parties to derive $sk := \sum_{j \in [n]} v_j$.

(3) Simulate the honest parties and $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dlc}}}$ to correctly compute (sk_i, D_i) for the honest parties while extracting the corrupted parties' shares $\{sk_j\}_{j \in \mathbf{P}^*}$.

(4) When releasing $\{(H_i, X_i)\}_{i \in [n]}$ to construct $(\mathbf{H}, X)$, sample $(\mathbf{H}_i, X_i)$ correctly for honest indices $i \neq k$, and set $(\mathbf{H}_k := \mathbf{H} - \sum_{j \in [n] \wedge j \neq k} \mathbf{H}_j,\ X_k :=$

$X - \sum_{j\in[n]\wedge j\neq k} X_j$) by manipulating $\mathcal{F}_{\mathsf{COM}}$, ensuring that the resulting verification key matches the expected value. When releasing $\{(\phi_{K_i}, \overline{x_i^{K_i}})\}_{i\in[n]}$ to construct $(\phi_K, \overline{x^K})$, correctly compute $(\phi_{K_i}, \overline{x_i^{K_i}})$ for all honest indices $i \neq k$, and set $(\phi_{K_k} := \mathfrak{g}^{r_k},\ \overline{x_k^{K_k}} := \phi_{K_k}^{sk}\mathfrak{f}^{\tilde{x}_k})$ with $r_k \leftarrow \mathcal{D}'$ and $\tilde{x}_k \xleftarrow{\$} \mathbb{Z}_q$.

Simulating signing: Let $\mathcal{T}^*$ denote the set of corrupted signer indices and $\overline{\mathcal{T}^*} := \mathcal{T} \setminus \mathcal{T}^*$ the set of honest signer indices. $\mathcal{S}$ receives (A, e, s) from $\mathcal{F}_{\mathsf{BBS+}}$, fixes an arbitrary new honest index $k \in \overline{\mathcal{T}^*}$, and proceeds as follows:

(5) Broadcast $\phi_{K^{\gamma_i}} := \phi_K^{\gamma_i} \in \langle \mathfrak{g}_{\mathfrak{q}} \rangle$ for $\gamma_i \leftarrow \mathcal{D}$ for every honest index i, while extracting γ_j in the form (γ'_j, ρ_j) using $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}}$ for all corrupted indices j. ϕ_{K^γ} is then computed as $\prod_{j\in\mathcal{T}} \phi_{K^{\gamma_j}}$, consistent with the real protocol.

(6) When releasing $\{(e_i, s_i)\}_{i\in\mathcal{T}}$ to construct (e, s), obtain (e, s) from $\mathcal{F}_{\mathsf{BBS+}}$ and sample (e_i, s_i) correctly for all honest indices $i \neq k$. Then manipulate $\mathcal{F}_{\mathsf{COM}}$ to set $e_k := e - \sum_{j\in[n]\wedge j\neq k} e_j$ and $s_k := s - \sum_{j\in[n]\wedge j\neq k} s_j$, ensuring that the resulting (e, s) match the expected values. Finally, use s to compute $B = G_1 + s \cdot H_1 + \sum_{i\in[\ell]} m_i \cdot H_i$.

(7) For every corrupted index $j \in \mathcal{T}^*$, play as $\mathcal{F}_{\mathsf{zero}}$ to distribute α_j to P_j, then deduce[10] the values sk'_j, the partial decryption pd_j, and the scaled elliptic-curve point R_j as $sk'_j := \Delta L_j^{\mathcal{T}}(0) \cdot sk_j$, $\overline{\gamma_j(x+e)^{K^{\gamma_j}}} := \left(\overline{x^K}\mathfrak{f}^e\right)^{\gamma'_j 2^{-\rho_j}}$, $pd_j := \frac{\overline{\gamma_j(x+e)^{K^{\gamma_j}}}^{\Delta^2}}{\phi_{K^\gamma}^{sk'_j} \cdot \mathfrak{f}^{\alpha_j}}$, and $R_j := (\gamma'_j \cdot 2^{-\rho_j}) \cdot B$.

(8) Given A from $\mathcal{F}_{\mathsf{BBS+}}$, correctly generate (pd_i, R_i) for all honest indices $i \neq k$ according to the protocol, while computing $pd_k := \frac{\mathfrak{f}^{\Delta^2 u}}{\prod_{j\in\mathcal{T}\setminus\{k\}} pd_j}$ and $R_k := u \cdot A - \sum_{j\in\mathcal{T}\setminus\{k\}} R_j$ for the honest index k, with $u \xleftarrow{\$} \mathbb{Z}_q$. Broadcast $\{(pd_i, R_i)\}_{i\in\overline{\mathcal{T}^*}}$ for honest parties and receive $\{(pd_j^*, R_j^*)\}_{j\in\overline{\mathcal{T}^*}}$ from corrupted parties. Derive the signature using these values and abort if the final-round verification fails.

The simulation now concludes. We proceed with a sequence of hybrid games to bound the gap between the real-world and ideal-world distributions. Each hybrid's distribution is defined as the joint distribution of the adversary $\mathcal{A}$'s view and the honest parties' outputs, and we let $\mathcal{H}_0$ denote the real-world experiment.

Hybrid $\mathcal{H}_1$. In $\mathcal{H}_1$, we introduce a simulator $\mathcal{S}'$ that (i) emulates all honest parties and hybrid functionalities by faithfully executing their prescribed code in interactions with other entities, and (ii) replaces every proof generated by honest parties via $\Pi_{\mathcal{R}}.\mathsf{Prv}(\cdot,\cdot)$ with a simulated proof obtained through $\Pi_{\mathcal{R}}.\mathsf{Sim}(\cdot)$ for any relation $\mathcal{R}$. The simulator $\mathcal{S}'$ further intercepts any values sent by corrupted parties to honest parties or to ideal functionalities. After each subprotocol completes, $\mathcal{S}'$ computes all outputs according to the prescribed execution and delivers them to the corresponding honest parties.

[10] "Deduce" here means extracting the expected value (in the case that the corrupted party behaves honestly) using known information.

The differences between $\mathcal{H}_1$ and $\mathcal{H}_0$ are purely syntactic or computationally bounded by the Zero-Knowledge (HVZK) property, and hence $\mathcal{H}_1 \approx_c \mathcal{H}_0$.

Hybrid $\mathcal{H}_2$. In $\mathcal{H}_2$, we modify the signing-phase behavior of $\mathcal{S}'$. Specifically, $\mathcal{S}'$ fixes an arbitrary $k \in \overline{\mathcal{T}^*}$, computes A after generating (e, s), deduces the expected values $\{(sk_j, pd_j, R_j)\}_{j\in\mathcal{T}^*}$ for corrupted parties, and derives P_k's (pd_k, R_k) according to steps (7) and (8) of $\mathcal{S}$. The sole difference between $\mathcal{H}_2$ and $\mathcal{H}_1$ lies in the distribution of (pd_k, R_k).

We claim these distributions are statistically close. Corrupted parties learn only $\gamma_k \bmod s'$ from $\phi_{K^{\gamma_k}} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$, which reveals negligible information about $\gamma_k \bmod q$ for $\gamma_k \leftarrow \mathcal{D}$. Consequently, $R_i = \gamma_i \cdot B$ is (close to) uniformly distributed over $\mathbb{G}_1$ in $\mathcal{H}_1$, and the same uniformity holds in $\mathcal{H}_2$ due to $u \xleftarrow{\$} \mathbb{Z}_q$. The distribution of pd_k is immaterial, since in both hybrids it is uniquely determined by A (itself fixed by (e, s)) and all other parties' expected values $\{(pd_i, R_i)\}_{i\neq k}$. Therefore, $\mathcal{H}_2 \approx_s \mathcal{H}_1$.

Hybrid $\mathcal{H}_3$. In $\mathcal{H}_3$, we modify the key-generation behavior of $\mathcal{S}'$. Specifically, $\mathcal{S}'$ fixes some $k \in \overline{\mathbf{P}^*}$, samples the honest parties' secrets $\{v_i\}_{i\in\overline{\mathbf{P}^*}}$, and computes the associated values following step (2) of $\mathcal{S}$. The differences between $\mathcal{H}_3$ and $\mathcal{H}_2$ are:

(1) Computation of $C_{k,0}$ (contributing to pk) due to different sampling of v_k;
(2) Computation of the corrupted parties' shares $\{y_{k,j}\}_{j\in\mathbf{P}^*}$ and values $\{C_{k,p}\}_{p=1}^{t-1}$;
(3) Computation of the honest parties' shares $\{(y_{k,i}, sk_i)\}_{i\in\overline{\mathbf{P}^*}}$.

Denote by $v_{\mathcal{H}_\beta}$ the value of v_k in $\mathcal{H}_\beta$ for $\beta \in \{2, 3\}$, so that $v_{\mathcal{H}_2} \leftarrow \mathcal{D}_q$ and $v_{\mathcal{H}_3} \leftarrow \mathcal{D}$. By the properties of $\mathcal{D}_q$ and $\mathcal{D}$, $C_{k,0} = {\mathfrak{g}_\mathfrak{q}}^{v_k}$ is statistically close to uniform in $\langle \mathfrak{g}_\mathfrak{q} \rangle$ regardless of whether $v_k = v_{\mathcal{H}_2}$ or $v_k = v_{\mathcal{H}_3}$. Hence, we may assume identical $C_{k,0}$ (tackling difference (1)) and pk in both hybrids and focus on $v_{\mathcal{H}_2} \in \mathcal{D}_q$ and $v_{\mathcal{H}_3} \in \mathcal{D}$ satisfying ${\mathfrak{g}_\mathfrak{q}}^{v_{\mathcal{H}_2}} = C_{k,0} = {\mathfrak{g}_\mathfrak{q}}^{v_{\mathcal{H}_3}}$, which implies

$$v_{\mathcal{H}_2} \equiv v_{\mathcal{H}_3} \bmod s'. \tag{2}$$

For difference (2), the privacy lemma ensures the statistical closeness of $\{y_{k,j}\}_{j\in\mathbf{P}^*}$ between $\mathcal{H}_2$ and $\mathcal{H}_3$; given consistent corrupted-party shares, Feldman's simulation yields identical $\{C_{k,p}\}_{p=1}^{t-1}$ in both hybrids.

For difference (3), let $y_{\mathcal{H}_\beta,i}$ and $sk_{\mathcal{H}_\beta,i}$ denote the share and secret in $\mathcal{H}_\beta$ ($\beta \in \{2, 3\}$, $i \in \overline{\mathbf{P}^*}$). Lagrange interpolation gives

$$y_{\mathcal{H}_2,i} = \frac{1}{\Delta}(\Delta L_0^{0\cup\mathbf{P}^*}(i) \cdot \Delta v_{\mathcal{H}_2} + \sum_{j\in\mathbf{P}^*} \Delta L_j^{0\cup\mathbf{P}^*}(i) \cdot y_{k,j})$$

$$y_{\mathcal{H}_3,i} = \frac{1}{\Delta}(\Delta L_0^{0\cup\mathbf{P}^*}(i) \cdot \Delta v_{\mathcal{H}_3} + \sum_{j\in\mathbf{P}^*} \Delta L_j^{0\cup\mathbf{P}^*}(i) \cdot y_{k,j})$$

so that $y_{\mathcal{H}_3,i} - y_{\mathcal{H}_2,i} = \Delta L_0^{\mathbf{P}^*\cup\{0\}}(i) \cdot (v_{\mathcal{H}_3} - v_{\mathcal{H}_2})$. Together with (2), this ensures $y_{\mathcal{H}_3,i} \equiv y_{\mathcal{H}_2,i} \bmod s'$, and therefore $sk_{\mathcal{H}_3,i} \equiv sk_{\mathcal{H}_2,i} \bmod s'$. Since sk_i is used only in signing to compute $pd_i := \frac{\left(\overline{\gamma_i(x+e)^{K^{\gamma_i}}}\right)^{\Delta^2}}{\phi_{K^{\gamma}}^{sk'_i}}$, where $sk'_i := \Delta L_i^{\mathcal{T}}(0) \cdot sk_i$ and $\phi_{K^\gamma} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$, this congruence implies identical pd_i values in both hybrids.

Collectively, these arguments establish $\mathcal{H}_3 \approx_s \mathcal{H}_2$.

Hybrid $\mathcal{H}_4$. In $\mathcal{H}_4$, we further modify the key-generation behavior of $\mathcal{S}'$ by sampling the randomness r_k used to generate ϕ_{K_k} uniformly from $\mathcal{D}'$ rather than $\mathcal{D}_q$. Since $K_k = pk^{r_k} = \phi_{K_k}^{sk}$, $\mathcal{S}'$ now computes K_k directly as $\phi_{K_k}^{sk}$ without using r_k.

The methods for computing K_k in $\mathcal{H}_3$ and $\mathcal{H}_4$ are equivalent, so the only difference arises from the distribution of $\phi_{K_k} = {\mathfrak{g}_\mathfrak{q}}^{r_k} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$. The statistical distance between sampling from $\mathcal{D}'$ and $\mathcal{D}_q$ modulo s' is bounded by $2^{-\lambda}+2^{-\lambda_d} \leq 2^{-\lambda_d+1}$, which is negligible in λ_d. As $\phi_{K_k} \in \langle \mathfrak{g}_\mathfrak{q} \rangle$ and $|\langle \mathfrak{g}_\mathfrak{q} \rangle| = s'$, its resulting distributions are statistically indistinguishable in $\mathcal{H}_4$ and $\mathcal{H}_3$. Hence, $\mathcal{H}_4 \approx_s \mathcal{H}_3$.

Hybrid $\mathcal{H}_5$. In $\mathcal{H}_5$, we further modify the key-generation behavior of $\mathcal{S}'$: it now generates ϕ_{K_k} as $\mathfrak{g}^{r_k}$ instead of ${\mathfrak{g}_\mathfrak{q}}^{r_k}$.

Since $r_k \leftarrow \mathcal{D}'$, the HSM assumption implies that $\mathfrak{g}^{r_k}$ and ${\mathfrak{g}_\mathfrak{q}}^{r_k}$ are computationally indistinguishable, and therefore $\mathcal{H}_5 \approx_c \mathcal{H}_4$.

Hybrid $\mathcal{H}_6$. In $\mathcal{H}_6$, we further modify the key-generation behavior of $\mathcal{S}'$: it now computes $\overline{x_k^{K_k}} := \phi_{K_k}^{sk} \mathfrak{f}^{\tilde{x}_k}$ following step (4) of $\mathcal{S}$, thereby encrypting a random value $\tilde{x}_k \xleftarrow{\$} \mathbb{Z}_q$, whereas in $\mathcal{H}_5$ we have $\overline{x_k^{K_k}} = \mathsf{Enc}(K_k, x_k) = \phi_{K_k}^{sk} \mathfrak{f}^{x_k}$.

By the smoothness lemma, given $\phi_{K_k} = \mathfrak{g}^{r_k} \in G \setminus \langle \mathfrak{g}_\mathfrak{q} \rangle$ with probability $1 - \frac{1}{q}$ and $C_{k,0} = {\mathfrak{g}_\mathfrak{q}}^{v_k}$ for $v_k \leftarrow \mathcal{D}$, the distributions of $\phi_{K_k}^{v_k} \mathfrak{f}^{x_k}$ and $\phi_{K_k}^{v_k} \mathfrak{f}^{\tilde{x}_k}$ are statistically indistinguishable. Let $\delta_k := sk - v_k = \sum_{i \in [n],\, i \neq k} v_i$. It follows that

$$\overline{x_k^{K_k}} = \phi_{K_k}^{sk} \mathfrak{f}^{x_k} = \phi_{K_k}^{\delta_k} \cdot \phi_{K_k}^{v_k} \mathfrak{f}^{x_k}$$

of $\mathcal{H}_5$ is statistically indistinguishable from

$$\overline{x_k^{K_k}} = \phi_{K_k}^{sk} \mathfrak{f}^{x_k} = \phi_{K_k}^{\delta_k} \cdot \phi_{K_k}^{v_k} \mathfrak{f}^{\tilde{x}_k}$$

of $\mathcal{H}_6$. Now, let $sk'_{\mathcal{H}_\beta,i}$ denote the value of sk'_i in hybrid $\mathcal{H}_\beta$ for $\beta \in \{5,6\}$ and $i \in \overline{\mathcal{T}^*} \setminus \{k\}$, where ϕ_{K_k}, $\overline{x_k^{K_k}}$, and $C_{k,0}$ remain identical between $\mathcal{H}_5$ and $\mathcal{H}_6$. Following analogous reasoning to $\mathcal{H}_3$, we conclude that $sk'_{\mathcal{H}_5,i} \equiv sk'_{\mathcal{H}_6,i} \bmod s'$. However, $sk'_{\mathcal{H}_5,i} \not\equiv sk'_{\mathcal{H}_6,i} \bmod q$. When decomposing $\phi_{K^\gamma}^{sk'_{\mathcal{H}_\beta,i}}$ (computed during pd_i evaluation) as ${\mathfrak{g}_\mathfrak{q}}^{a_{\beta,i}} \mathfrak{f}^{b_{\beta,i}}$ for $\beta \in \{5,6\}$, this implies $a_{5,i} = a_{6,i}$ but $b_{5,i} \neq b_{6,i}$. Luckily, multiplying $\phi_{K^\gamma}^{sk'_{\mathcal{H}_\beta,i}}$ by $\mathfrak{f}^{\alpha_i}$ solves this concern. The uniformness of α_i ensures that $b_{5,i}+\alpha_i$ and $b_{6,i}+\alpha_i$ follow identical distributions, which establishes $\mathcal{H}_5 \approx_s \mathcal{H}_6$.

Hybrid $\mathcal{H}_7$. In $\mathcal{H}_7$, we modify the construction of $(\mathbf{H}, X)$ and (e, s) by $\mathcal{S}'$: these values are generated in advance, and $\mathcal{F}_{\mathsf{COM}}$ is then manipulated following $\mathcal{S}$'s procedure to ensure that the derived values coincide with the pre-sampled ones. This preserves an identical view for $\mathcal{A}$ in both hybrids, and therefore $\mathcal{H}_7 = \mathcal{H}_6$.

Hybrid $\mathcal{H}_8$. In $\mathcal{H}_8$, we modify $\mathcal{S}'$ so that $(\mathbf{H}, X)$ and (A, e, s) are obtained directly from $\mathcal{F}_{\mathsf{BBS+}}$. This change induces no distributional difference between $\mathcal{H}_8$ and $\mathcal{H}_7$, hence $\mathcal{H}_8 = \mathcal{H}_7$. Moreover, $\mathcal{H}_8$ coincides with the ideal-world distribution. Chaining the hybrids from $\mathcal{H}_0$ to $\mathcal{H}_8$ yields computational indistin-

guishability between the real-world and ideal-world distributions, completing the proof.

D Protocol Details of Threshold BBS Signatures and Ovlivious DY VRFs

In both our threshold BBS and oblivious DY VRF protocols, the setup of the CL KEM environment is performed during the first phase of key generation (designated as part I). This stage is identical to part I of the original threshold BBS+ key generation (see Fig. 6). Consequently, we focus on describing part II of the key generation for threshold BBS and oblivious DY VRFs, which involves Rounds 4 and 5, following a structure analogous to the part II of the threshold BBS+ key generation. These procedures are specified in Fig. 12. Additionally, the corresponding threshold BBS signing and DY VRF oblivious evaluation protocols are presented in Fig. 13 and Fig. 14, respectively.

Remark 7. Note that our implemented AGM-secure BBS targets the specification considered in Section 4.1, where a cryptographic hash $H(\cdot)$ is utilized to derive the nonce e. In this specification, there is no need to construct a public random state st_e during the key generation phase. If a specification with a public random state is desired, one can employ a commit-and-release paradigm (which can be integrated into the construction of (X, H) during key generation) to derive a random tape r^{st}. This tape is then used to locally compute the public st_e (which corresponds to the algorithm $\mathsf{eG}(\mathcal{G}; r^{\mathsf{st}})$ in [29]). Subsequently, during the actual threshold signing phase, the nonce e is derived either via $\mathsf{eS}(\mathsf{st}_e, \mathsf{cnt})$ (if a global signing counter cnt is synchronized) or via $\mathsf{eS}(\mathsf{st}_e, \mathcal{T} \parallel \mathsf{cnt}_{\mathcal{T}})$ (if a specific counter $\mathsf{cnt}_{\mathcal{T}}$ is maintained for each signing index set $\mathcal{T}$).

Setup for the verifying key and related components under the session id sid.

Every party P_i, where $i \in [n]$ **Parties** $\{P_j\}_{j\in[n]\setminus\{i\}}$

... **Setup the verifying key, encrypt x and release the key encapsulation** ...

Round 4:

$(\phi_{K_i}, K_i) \leftarrow \mathsf{Encap}(pk; r_i),\ x_i \xleftarrow{\$} \mathbb{Z}_q,\ \overline{x_i^{K_i}} \leftarrow \mathsf{Enc}(K_i, x_i)$

For BBS and its AGM-secure variant:

$$X_i := x_i \cdot G_2,\ \mathbf{H}_i \xleftarrow{\$} \mathbb{G}_1^{\ell} \xrightarrow{(\mathsf{commit},\mathrm{sid},[n],(\phi_{K_i},\overline{x_i^{K_i}},\mathbf{H}_i,X_i))} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{commit},\ldots)}$$

For oblivious DY VRFs:

$$X_i := x_i \cdot G_2, \xrightarrow{(\mathsf{commit},\mathrm{sid},[n],(\phi_{K_i},\overline{x_i^{K_i}},X_i))} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{commit},\ldots)}$$

Round 5: $\xrightarrow{(\mathsf{decommit},\mathrm{sid})} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{decommit},\ldots)}$

$\pi_i \leftarrow \Pi_{\mathcal{R}_{\mathsf{enc}}}.\mathsf{Prv}((pk, \phi_{K_i}, \overline{x_i^{K_i}}, X_i), (x_i, r_i))$

$$\xrightarrow[\text{Broadcast to all parties}]{(\mathsf{zk\text{-}verify1},\mathrm{sid},i,\pi_i)}$$

If $\Pi_{\mathcal{R}_{\mathsf{enc}}}.\mathsf{Vrf}((pk, \phi_{K_i}, \overline{x_i^{K_i}}, X_i), \pi_i) = 0$, **abort**

Compute and store $\phi_K = \prod_{j\in[n]} \phi_{K_j},\ \overline{x^K} \leftarrow \prod_{j\in[n]} \overline{x_j^{K_j}},\ X := \sum_{j\in[n]} X_j$

For BBS and its AGM-secure variant:

Compute and store $\mathbf{H} := \sum_{j\in[n]} \mathbf{H}_j$, then **output** $(\mathsf{vk}, \mathrm{sid}, (X, \mathbf{H}))$

For oblivious DY VRFs:

Output $(\mathsf{vk}, \mathrm{sid}, X)$

Fig. 12: Threshold BBS/oblivious DY VRF key generation protocol part II.

Threshold signing by parties indexed by $\mathcal{T}$ on message $\mathbf{m} \in \mathbb{Z}_q^\ell$ and the session id sid.

Every party P_i, where $i \in \mathcal{T}$ **Parties** $\{P_j\}_{j \in \mathcal{T} \setminus \{i\}}$

Commit/Derive e_i, encrypt $\gamma_i x$ and construct a common encapsulation

Round 1:

For the original BBS:

$e_i \xleftarrow{\$} \mathbb{Z}_q$ $\xrightarrow{(\mathsf{commit}, \mathrm{sid}, \mathcal{T}, e_i)} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{commit}, \ldots)}$

For the AGM-secure BBS:

If $\mathsf{cnt}_\mathcal{T}$ does not exists in memory, initialize and store $\mathsf{cnt}_\mathcal{T} := 0$

$\mathsf{cnt}_\mathcal{T} = \mathsf{cnt}_\mathcal{T} + 1,\ e \leftarrow H(\mathcal{T} || \mathsf{cnt}_\mathcal{T})$

$\gamma_i \leftarrow \mathcal{D},\ \phi_{K^{\gamma_i}} := \phi_K^{\gamma_i},\ \overline{\gamma_i x^{K^{\gamma_i}}} \leftarrow \left(\overline{x^K}\right)^{\gamma_i}$ $\xrightarrow{(\mathsf{prove}, \mathrm{sid}, \mathcal{T}, (\phi_{\gamma_i}, \phi), (\gamma_i, 0))} \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathrm{dls}}} \xrightarrow{(\mathsf{proof}, \ldots, b_{\mathrm{sid}})}$

$\phi_{K^\gamma} := \prod_{j \in \mathcal{T}} \phi_{K^{\gamma_j}}$ If $b_{\mathrm{sid}} = 0$, **abort**

Open e_i for original BBS, threshold-decrypt $\gamma(x+e)$ and derive the signature

Round 2 of the original BBS: $\xrightarrow{(\mathsf{decommit}, \mathrm{sid})} \mathcal{F}_{\mathsf{COM}} \xrightarrow{(\mathsf{decommit}, \ldots, e_i)}$

$e := \sum_{j \in \mathcal{T}} e_j$

Round 3 of the original BBS or Round 2 of the AGM-secure BBS:

$B := G_1 + \sum_{k \in [\ell]} m_k \cdot H_k,\ R_i = \gamma_i \cdot B,$

$\overline{\gamma_i (x+e)^{K^{\gamma_i}}} \leftarrow \overline{\gamma_i x^{K^{\gamma_i}}} \cdot \overline{\gamma_i e^1}$ (Note that $\overline{\gamma_i e^1} = \mathfrak{f}^{\gamma_i e}$),

Input $(\mathsf{sample}, \mathrm{sid}, \mathcal{T})$ to $\mathcal{F}_{\mathsf{zero}}$ and obtain α_i

$sk_i' := \Delta L_i^\mathcal{T}(0) \cdot sk_i,\ pd_i := \dfrac{\left(\overline{\gamma_i (x+e)^{K^{\gamma_i}}}\right)^{\Delta^2}}{\phi_{K^\gamma}^{sk_i'} \cdot \mathfrak{f}^{\alpha_i}}$ $\xrightarrow[\text{Broadcast to signing parties indexed by } \mathcal{T}]{(\mathsf{part\text{-}sig}, \mathrm{sid}, i, R_i, pd_i)}$

$u' \leftarrow \mathsf{CL.Solve}(\prod_{j \in \mathcal{T}} pd_j)$: If $u' = \perp$, **abort**

Compute $u := u' \cdot \Delta^{-2} \bmod q$ and $A := \dfrac{\sum_{j \in \mathcal{T}} R_j}{u}$

If $\mathsf{e}(A, X + e \cdot G_2) \neq \mathsf{e}(B, G_2)$, **abort**

Otherwise, **output** $(\mathsf{signature}, \mathrm{sid}, (A, e))$

Fig. 13: Threshold BBS signing protocol.

Threshold evaluation by parties indexed by $\mathcal{T}$ and the session id sid.

User $\mathcal{U}$ with input e — **Broadcast the blind ciphertext** — **Parties $\{P_j\}_{j\in\mathcal{T}}$**

Sample $\eta \xleftarrow{\$} \mathcal{D}$ and compute $\phi_{K^\eta} := \phi_K^\eta, \overline{\eta(x+e)^{K^\eta}} := \left(\overline{x^K} \cdot \overline{e^1}\right)^\eta$

$$\xrightarrow{(\mathsf{prove},\mathrm{sid},\mathcal{T},(\phi_K,\overline{x^K},\phi_{K^\eta},\overline{\eta(x+e)^{K^\eta}}),(\eta,\eta e))} \mathcal{F}_{\mathsf{ZK}}^{\mathcal{R}_{\mathsf{scal\text{-}add}}} \xrightarrow{(\mathsf{proof},\ldots,b_{\mathrm{sid}})}$$

If $b_{\mathrm{sid}} = 0$, **abort**

Every party P_i, where $i \in \mathcal{T}$ — **Parties $\{P_j\}_{j\in\mathcal{T}\setminus\{i\}}$**

Encrypt $\gamma_i\eta(x+e)$ and broadcast related components

Round 1:

$\gamma_i \leftarrow \mathcal{D},\ \phi_{K^{\gamma_i\eta}} := \phi_{K^\eta}^{\gamma_i},\ \overline{\gamma_i\eta(x+e)^{K^{\gamma_i\eta}}} \leftarrow \left(\overline{\eta(x+e)^{K^\eta}}\right)^{\gamma_i}$

$\pi_i \leftarrow \Pi_{\mathcal{R}_{\mathrm{dls}}}.\mathsf{Prv}((\phi_{K^{\gamma_i\eta}}, \phi_{K^\eta}), \gamma_i)$ $\xrightarrow[\text{Broadcast to all evaluating parties indexed by } \mathcal{T}]{(\mathsf{zk\text{-}verify2},\mathrm{sid},i,\phi_{K^{\gamma_i\eta}},\pi_i)}$

If $\Pi_{\mathcal{R}_{\mathrm{dls}}}.\mathsf{Vrf}(\phi_{K^{\gamma_i\eta}}, \phi_{K^\eta}), \pi_i) = 0$, **abort**

$\phi_{K^{\gamma\eta}} := \prod_{j\in\mathcal{T}} \phi_{K^{\gamma_j\eta}}$

Every party P_i, where $i \in \mathcal{T}$ — **User $\mathcal{U}$**

Threshold-decrypt $\gamma\eta(x+e)$ and derive the VRF output

Round 2:

Compute $R_i = \gamma_i \cdot G_1$ and input $(\mathsf{sample}, \mathrm{sid}, \mathcal{T})$ to $\mathcal{F}_{\mathsf{zero}}$ to obtain α_i

$sk_i' := \Delta L_i^{\mathcal{T}}(0) \cdot sk_i,\ pd_i := \dfrac{\left(\overline{\gamma_i\eta(x+e)^{K^{\gamma_i\eta}}}\right)^{\Delta^2}}{\phi_{K^{\gamma\eta}}^{sk_i'} \cdot \mathfrak{f}^{\alpha_i}}$ $\xrightarrow[\text{Send to } \mathcal{U}]{(\mathsf{part\text{-}output},\mathrm{sid},i,R_i,pd_i)}$

$u' \leftarrow \mathsf{CL.Solve}(\prod_{j\in\mathcal{T}} pd_j)$: If $u' = \perp$, **abort**

Compute $u := u' \cdot \Delta^{-2} \cdot \eta^{-1} \bmod q$ and $A := \frac{\sum_{j\in\mathcal{T}} R_j}{u}$

If $\mathsf{e}(A, X + e \cdot G_2) \neq \mathsf{e}(G_1, G_2)$, **abort**

Otherwise, **output** $(\mathsf{vrf\text{-}output}, \mathrm{sid}, \mathsf{e}(A, G_2))$

Fig. 14: Threshold DY VRF oblivious evaluation protocol.